

CKA: Security

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Security Section Introduction

This article provides a comprehensive guide on Kubernetes security, covering access control, authentication mechanisms, TLS certificates, and authorization methods.

Hello, and welcome to this comprehensive article on Kubernetes security. My name is Mumshad Mannambeth, and in this guide, we will begin by exploring the security primitives of Kubernetes. We will explain how access is granted to a Kubernetes cluster, how various actions are controlled, and review the different authentication mechanisms available. Additionally, we will examine the default configurations of a cluster and demonstrate how to view the configurations of an existing cluster.

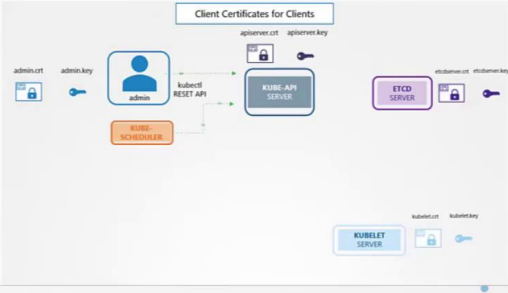
Below is an example CSV file containing user details:

```
csv      theme={null}      user-details.csv      password123,user1,u0001      password123,user2,u0002
password123,user3,u0003 password123,user4,u0004 password123,user5,u0005
```

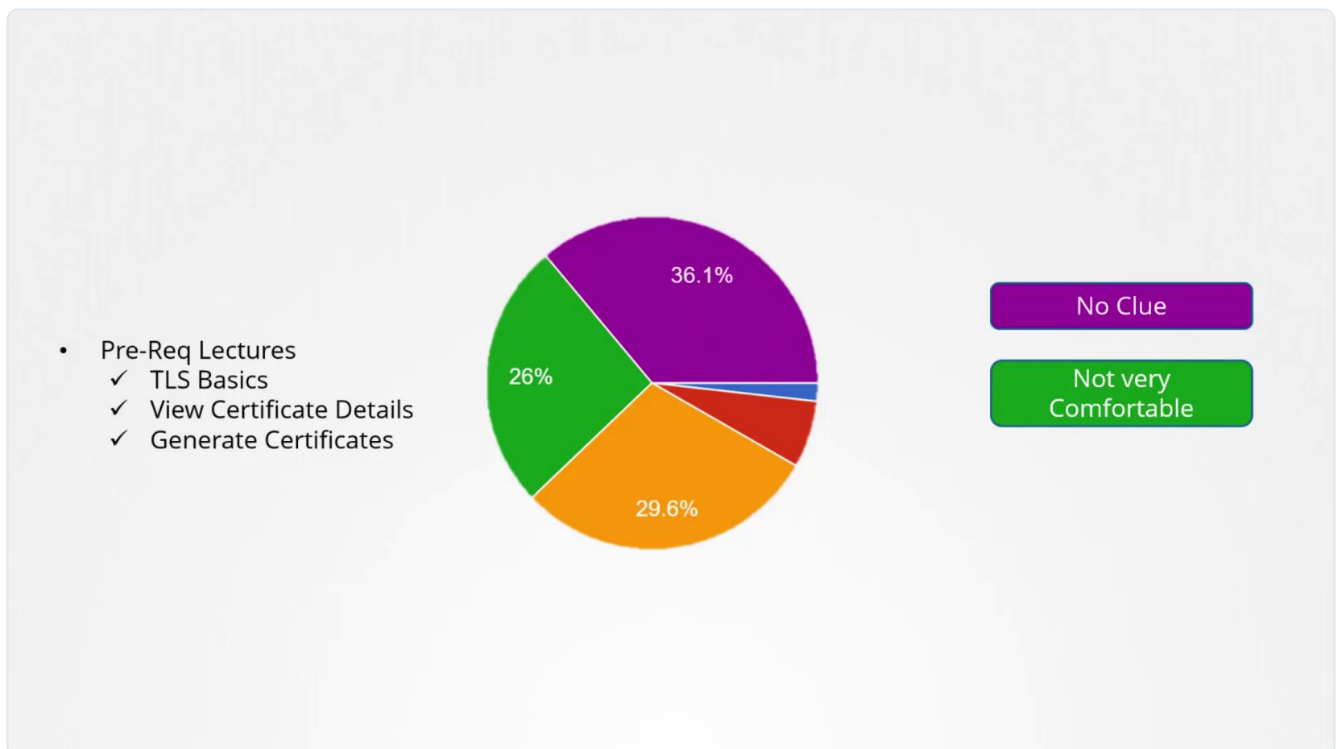
We then discuss TLS certificates and the role they play in securing components within the cluster. If you are a Kubernetes administrator setting up a cluster on your own, you might encounter challenges related to certificate management. This article covers the basics of certificates from the ground up.

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
 - ☐ Kubernetes Security Primitives
 - ☐ Authentication
 - ☐ TLS Certificates for Cluster Components
 - ☐ Secure Persistent Key Value Store
 - ☐ Authorization
 - ☐ Images Securely
 - ☐ Security Contexts
 - ☐ Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting



One of the key sections in this article simplifies the core concepts of certificates. For beginners, several prerequisite lectures have been included to build a strong foundation. A poll conducted among the audience revealed that most respondents were new to TLS certificates. This valuable feedback prompted a detailed explanation of how certificates are used within Kubernetes, a topic that often confuses newcomers.



The prerequisite lectures and accompanying practice tests are designed to help you bridge any gaps in your understanding of TLS certificates. Depending on your level of expertise, you might choose to skip some of these sessions.

Later in this article, we will dive into authorization with a specific focus on various mechanisms, including role-based access control (RBAC). We will also cover essential topics such as securing container images, implementing security contexts, and configuring network policies. There is a wealth of lectures and practice tests ahead, so take your time and enjoy the learning process.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Kubernetes Security Primitives

This article explores core security features essential for protecting production-grade Kubernetes clusters, including securing hosts, API server access control, and network policies.

Hello, and welcome to this lesson on Kubernetes security primitives. In this session, we explore the core security features essential for protecting production-grade Kubernetes clusters. This overview introduces key security measures before we delve into detailed configurations and best practices.

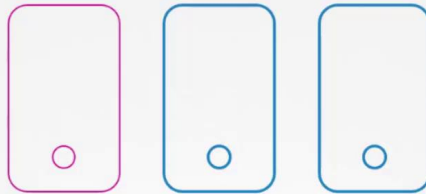
Securing Cluster Hosts

The security of your Kubernetes cluster begins with the hosts themselves. Protect your underlying infrastructure by following these practices:

- Disable root access.
- Turn off password-based authentication.
- Enforce SSH key-based authentication.
- Implement additional measures to secure your physical or virtual systems.

A compromised host can expose your entire cluster, so securing these systems is a critical first step.

Secure Hosts



- ☐ Password based authentication disabled
- ☐ SSH Key based authentication

API Server Access Control

The Kube API server is at the heart of Kubernetes operations because all cluster interactions—whether via the `kubectl` command-line tool or directly through API calls—pass through it. Effective access control is essential, focusing on two key questions:

1. Who can access the cluster?
2. What actions are they permitted to perform?

Authentication

Authentication verifies the identity of a user or service before granting access to the API server. Kubernetes offers various authentication mechanisms to suit different security needs:

- Static user IDs and passwords
- Tokens
- Client certificates
- Integration with external authentication providers (e.g., LDAP)

Additionally, service accounts support non-human processes. Detailed guidance on these methods is available in dedicated sections of our documentation.

| Authentication

Who can access?

- ☐ Files – Username and Passwords
- ☐ Files – Username and Tokens
- ☐ Certificates
- ☐ External Authentication providers - LDAP
- ☐ Service Accounts

Authorization

After authentication, authorization determines what actions a user or service is allowed to perform. The default mechanism, Role-Based Access Control (RBAC), associates identities with specific permissions. Kubernetes also supports:

- Attribute-Based Access Control (ABAC)
- Node Authorization
- Webhook-based authorization

These mechanisms enforce granular access control policies, ensuring that authenticated entities can perform only the operations they are permitted to execute.

| Authorization

What can they do?

- ☐ RBAC Authorization
- ☐ ABAC Authorization
- ☐ Node Authorization
- ☐ Webhook Mode

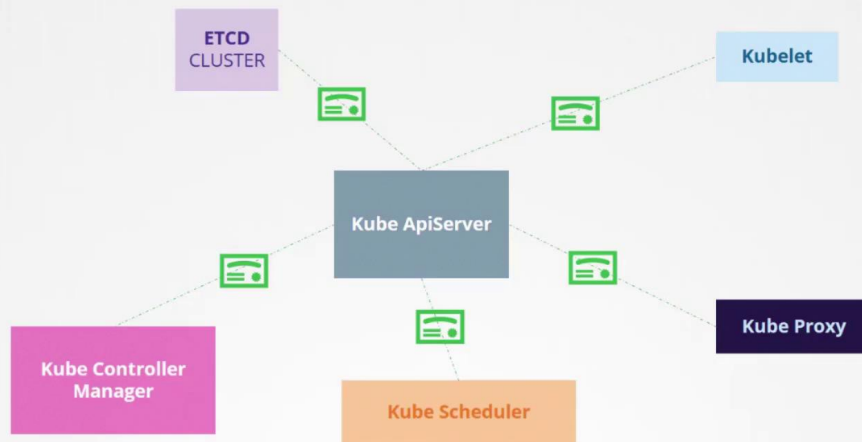
Securing Component Communications

Secure communications between Kubernetes components are enabled via TLS encryption. This ensures that data transmitted between key components remains confidential and tamper-proof. Encryption protects:

- Communication within the etcd cluster
- Interactions between the Kube Controller Manager and Kube Scheduler
- Links between worker node components such as the Kubelet and Kube Proxy

For more detailed instructions on setting up and managing TLS certificates, refer to our comprehensive certificate management guides.

| TLS Certificates



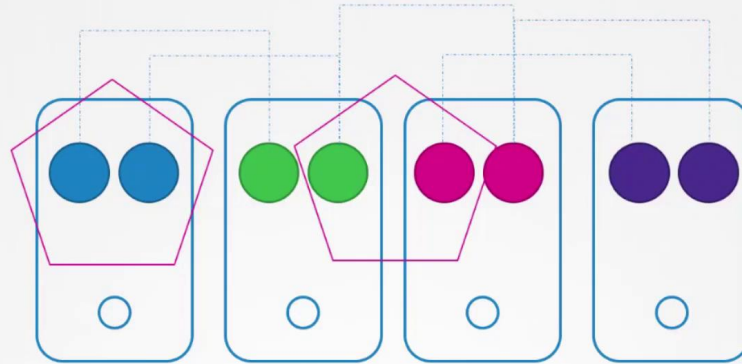
Network Policies

By default, pods in a Kubernetes cluster communicate freely with one another. To restrict unwanted interactions and enhance security, Kubernetes provides network policies. These policies allow you to:

- Control traffic flow between specific pods
- Enforce security rules at the network level

For an in-depth discussion on implementing network policies with practical examples, please see the related documentation.

Network Policies



For a secure Kubernetes environment, consider combining these security primitives with additional best practices and tools to counter evolving threats.

This high-level overview has introduced the critical security primitives in Kubernetes. For more detailed procedures and practical examples to further enhance the security of your clusters, please refer to the extended documentation.

Happy securing!

Documentation Index

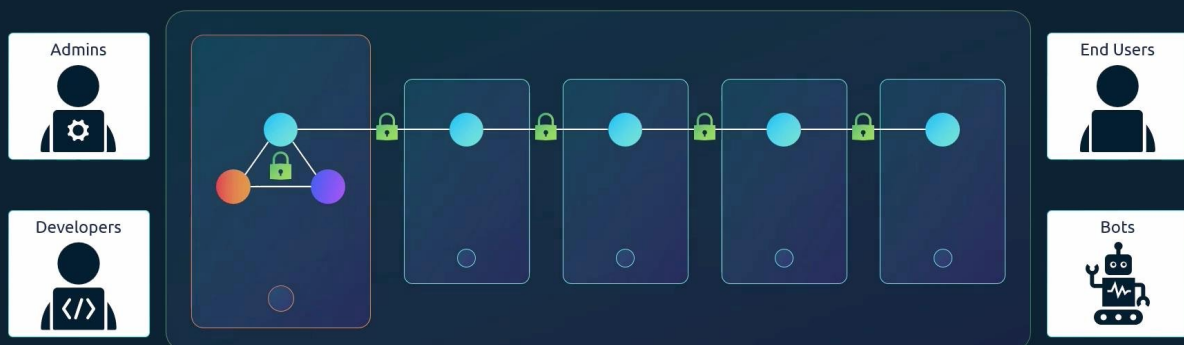
Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Authentication

Overview of Kubernetes authentication methods for kube-apiserver, covering legacy static files, TLS client certificates, external identity providers, service accounts, and operational recommendations.

Welcome to this lesson on authentication in a Kubernetes cluster. Kubernetes clusters run on multiple nodes (physical or virtual) and include components that coordinate access to the control plane and workloads. Several types of principals interact with the cluster:

- Administrators who perform cluster-level operations.
- Developers who deploy and iterate on applications.
- End users who access applications (application-level auth is handled by the apps themselves and is out of scope here).
- Robots (processes, controllers, CI systems, and third-party services) that call the Kubernetes API programmatically.



This lesson focuses on securing administrative access to the kube-apiserver — the central API endpoint that authenticates and authorizes all requests to the control plane. That includes access performed by humans (admins, developers) and machines (controllers, CI systems, operators).

Kubernetes does not manage regular user accounts natively: you cannot create or list standard user objects with `kubectl`. User identities are typically introduced to the cluster through external mechanisms, such as:

- static files (legacy),
- TLS client certificates,
- or an external identity provider (OIDC, LDAP, Kerberos, SAML, etc).

Service accounts, on the other hand, are a Kubernetes resource and are created/managed via the API. Example:

```
```bash theme={null}
```

## create a service account

---

```
kubectl create serviceaccount sa1
```

## list service accounts in the current namespace

---

```
kubectl get serviceaccounts
```

All incoming API requests (from kubectl, dashboard, controllers, or direct API calls) are received by kube-apiserver.

Below are the common authentication mechanisms you can configure on kube-apiserver.

```
<Frame>
```

```

```

```
</Frame>
```

#### Common authentication options

- \* Static basic-auth (password) files – legacy.
- \* Static token files – legacy.
- \* TLS client certificates – recommended for many production setups.
- \* External identity providers (OIDC, LDAP, Kerberos) – recommended for large or multi-tenant clusters.

Table: Authentication mechanisms at a glance

Mechanism	Use case	Example / Note
Static basic-auth file	Small experiments, local demos	CSV of users
Static token file	Simple automation, throwaway clusters	CSV mapping
TLS client certificates	Secure machine access, admin/user certificates	Use CA-signed
External identity providers (OIDC, LDAP)	Enterprise SSO, centralized user management	Integrate k

#### Legacy mechanisms: static basic-auth and token files

We'll briefly cover these two legacy mechanisms so you understand the underlying concepts. Both are simple to configure.

##### Static basic-auth file

- \* A basic-auth file is a CSV containing password, username, uid, and groups.
- \* kube-apiserver reads it when started with the `--basic-auth-file` flag.
- \* Credentials are stored in clear text – insecure and deprecated.

Example basic-auth CSV (legacy format):

```
```csv theme={null}
# password,username,uid,groups
KpjCVbI7rCFAHYPkByTIzRb7gu1cUc4B,user10,u0010,group1
rJjncHmvtXHc6MlWQddhtvNyvhgTdxSC,user11,u0011,group1
mjp0FIEiF0KL9toikaRNtt59ePtczZSq,user12,u0012,group2
PG41IXhs7QjqwWkmBkvgGT9g10yUqZij,user13,u0013,group2
```
```

Enable this mechanism on kube-apiserver (not recommended):

```
```bash theme={null} --basic-auth-file=/path/to/basic-auth.csv
```

Authenticate with HTTP Basic (example using curl):

```
```bash theme={null}
curl -k -u user10:KpjCVbI7rCFAHYPkByTIzRb7gu1cUc4B https://master-node-ip:6443/api
```

##### Static token file

- A token file is a CSV mapping bearer tokens to user identities and groups.
- kube-apiserver reads it via the `--token-auth-file` flag.

- Like the basic file, it stores tokens in plaintext — insecure and deprecated.

Example token CSV (legacy format):

```
```csv theme={null}
```

token,username,uid,groups

```
c29tZXRva2VuMTIz,user21,u0021,group1 aW5vdGhlcnRva2Vu,user22,u0022,group2
```

Enable on kube-apiserver (not recommended):

```
```bash theme={null}
--token-auth-file=/path/to/token-auth.csv
```

Authenticate with a bearer token (curl example):

```
bash theme={null} curl -k -H "Authorization: Bearer c29tZXRva2VuMTIz" https://master-node-
ip:6443/api
```

### Important operational notes

- kubeadm-managed clusters: kube-apiserver runs as a static pod. To add flags such as `--basic-auth-file` or `--token-auth-file` you must edit the kube-apiserver manifest (commonly `/etc/kubernetes/manifests/kube-apiserver.yaml`) and add `hostPath/volume` mounts so the files are accessible to the pod. The kubelet will detect the manifest change and restart kube-apiserver automatically.
- When you modify kube-apiserver flags or change its static pod manifest, the kube-apiserver process will restart under kubeadm-managed setups; plan for a short control-plane interruption.
- Always combine authentication with proper authorization (RBAC) to control what an authenticated identity can do.

Static basic-auth and token files store credentials in clear text and are considered legacy and unsafe. They are deprecated in modern Kubernetes releases. Prefer certificate-based authentication or external identity providers (OIDC, LDAP, etc.), and always enforce RBAC for authorization.

## Notes

This is not a recommended authentication mechanism

Consider volume mount while providing the auth file in a kubeadm setup

Set up Role-Based Authorization for new users

© Copyright KodeKloud

### Summary and recommendations

- kube-apiserver is the central authentication gateway for Kubernetes; every request to the API is authenticated by it.
- Kubernetes does not create or manage regular user accounts itself — integrate with external identity providers or use client certificates. Service accounts are native Kubernetes objects for in-cluster processes.
- Static basic-auth and token files are easy to understand but insecure and deprecated. Use them only for quick tests or isolated labs.
- For production, prefer TLS client certificates or integrate kube-apiserver with a robust external identity provider (OIDC, LDAP). Always pair authentication with RBAC authorization.

### Further reading and references

- [Kubernetes Authentication Overview](#)
- [kube-apiserver command line reference](#)
- [Using OpenID Connect \(OIDC\) with Kubernetes](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# TLS Introduction

This article provides a guide on understanding and managing TLS certificates in Kubernetes.

Hello, and welcome to this comprehensive guide on TLS certificates in Kubernetes.

Securing your Kubernetes cluster with TLS might seem challenging if you're not already familiar with TLS certificates. In developing this course, we surveyed our audience and discovered that many of you felt uncomfortable or had limited knowledge about TLS certificates. To address this, we've created a series of lessons designed to help you understand and manage certificates within Kubernetes—and beyond.

If you already have a solid understanding of TLS certificates, feel free to skip directly to the Kubernetes-specific sections. By the end of this article, you will be equipped with the skills to efficiently configure, monitor, and troubleshoot certificate-related issues in your Kubernetes environment, while also gaining a broader understanding of how TLS certificates work overall. Mastering these concepts will enhance your ability to secure and maintain robust infrastructures.

## Goals!

- ☐ What are TLS Certificates?
- ☐ How does Kubernetes use Certificates?
- ☐ How to generate them?
- ☐ How to configure them?
- ☐ How to view them?
- ☐ How to troubleshoot issues related to Certificates

We begin with the fundamentals of certificates, covering key concepts such as certificate authorities, issuance, and validation processes. If these basics are already within your grasp, you may proceed to the next section where we delve deeper into managing certificates specifically in Kubernetes.

For best results, it is recommended to review the foundational lessons on certificate management before tackling the Kubernetes-specific content.

## Documentation Index

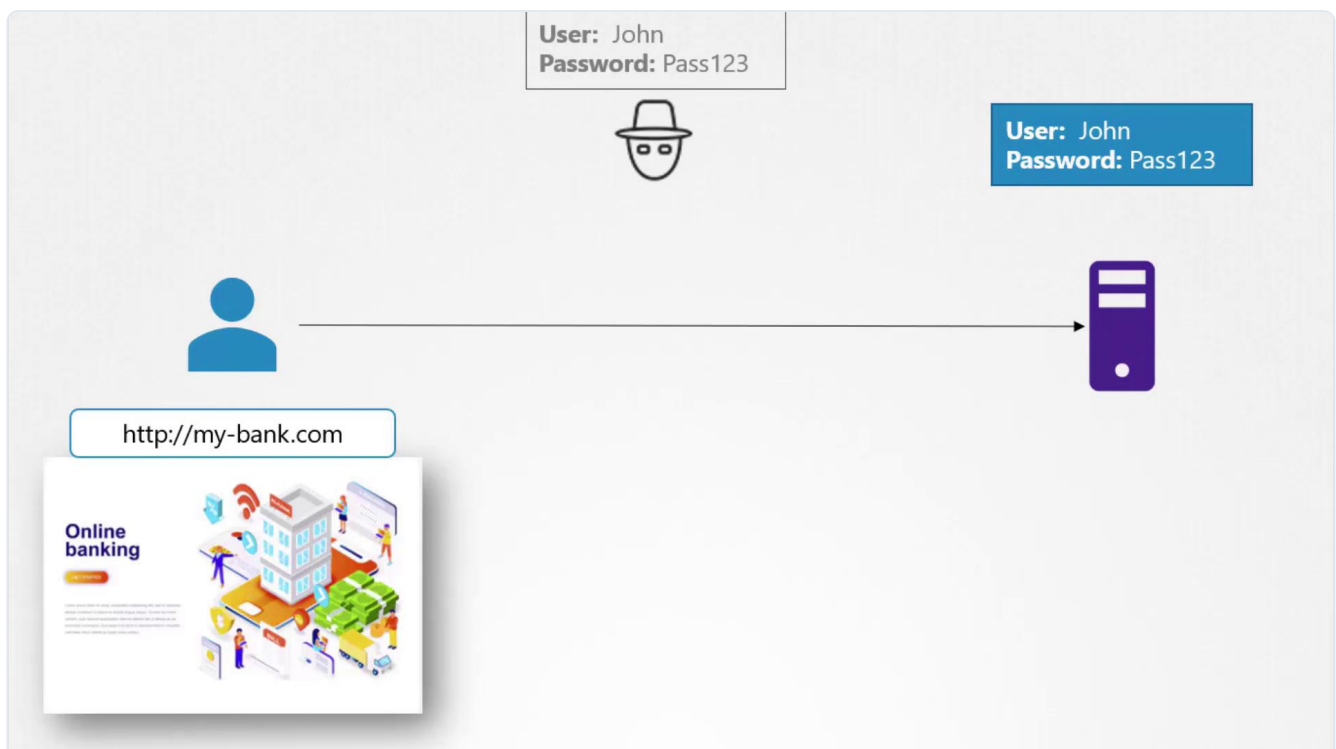
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# TLS Basics

This article covers the fundamentals of TLS certificates, their importance for secure communications, and configuration for SSH and web servers.

Welcome to this comprehensive lesson on SSL/TLS certificates. In this guide, we will explore the fundamentals of TLS certificates, discuss why they are essential for secure communications, and demonstrate how to configure them for securing SSH and web servers. A TLS certificate establishes trust during a transaction by ensuring that communications are encrypted and that the server is indeed who it claims to be.

Imagine a scenario where a user accesses their online banking application over an unsecured connection. The credentials entered would be transmitted in plain text, making it easy for a hacker spying on the network to intercept and misuse the sensitive data.



To prevent such risks, data is encrypted using encryption keys—a set of random numbers and characters. Initially, symmetric encryption was used, where the same key is responsible for both encrypting and decrypting data. However, transmitting this key over the network to initiate a secure session introduces vulnerabilities, as an attacker intercepting the key could decrypt the data.

This is where asymmetric encryption becomes valuable. Asymmetric encryption uses a pair of keys: a private key and a public key. You can think of these as a private key and a public lock. The private key remains securely with the owner, while the public lock can be shared openly. Data encrypted with the public key can only be decrypted using its corresponding private key, ensuring that intercepted data remains secure.

Before delving into the web server example, let's explore a simpler use case: securing SSH access using key pairs.



Imagine you need to access a server but want to avoid the security risks associated with passwords. By using key pairs, you can generate a private key (`id_rsa`) and a public key (`id_rsa.pub`). The private key stays secure on your device, while the public key is added to the server's SSH authorized keys file. When you initiate an SSH connection, you specify your private key to authenticate.

:::note SSH Key Pair Generation Generate your SSH key pair with the following command: :::

```
```bash theme={null}
```

Generate SSH key pair

ssh-keygen

Files generated: id_rsa (private key) and id_rsa.pub (public key)

You can review the authorized keys on the server as follows:

```
```bash theme={null}
cat ~/.ssh/authorized_keys
```

An example output might appear like:

```
```plaintext theme={null} ssh-rsa AAAAB3NzaC1yc...KhtUBfoTz1BqRV1NThvO0apzEwRQo1mWx user1
```

To connect securely to the server using your key pair, use:

```
```bash theme={null}
ssh -i id_rsa user1@server1
```

If you have multiple servers, simply copy your public key to each server's authorized keys file so you can authenticate using the same private key on all servers. For additional users who require access, they can generate their own key pairs and have their public keys added to the servers.

Now, let's return to the web server scenario. With symmetric encryption, the key used for encryption must be sent along with the ciphertext, which introduces risk if intercepted. Asymmetric encryption addresses this by securely transferring the symmetric key. Here's how the process works for a web server using HTTPS:

1. The server generates a key pair (private and public keys).
2. Upon a user's initial HTTPS request, the server sends its public key embedded within a certificate.
3. The client's browser encrypts a newly generated symmetric key using the server's public key.
4. The encrypted symmetric key is sent back to the server.
5. The server decrypts the symmetric key using its private key.
6. All subsequent communications are encrypted with this symmetric key.

For example, to generate a key pair with OpenSSL for encrypting the symmetric key, you can use:

```
```bash theme={null}
```

Generate a private key

```
openssl genrsa -out my-bank.key 1024
```

Extract the public key

```
openssl rsa -in my-bank.key -pubout > mybank.pem
```

The above commands demonstrate how to create the necessary keys. Although the original content repeated the

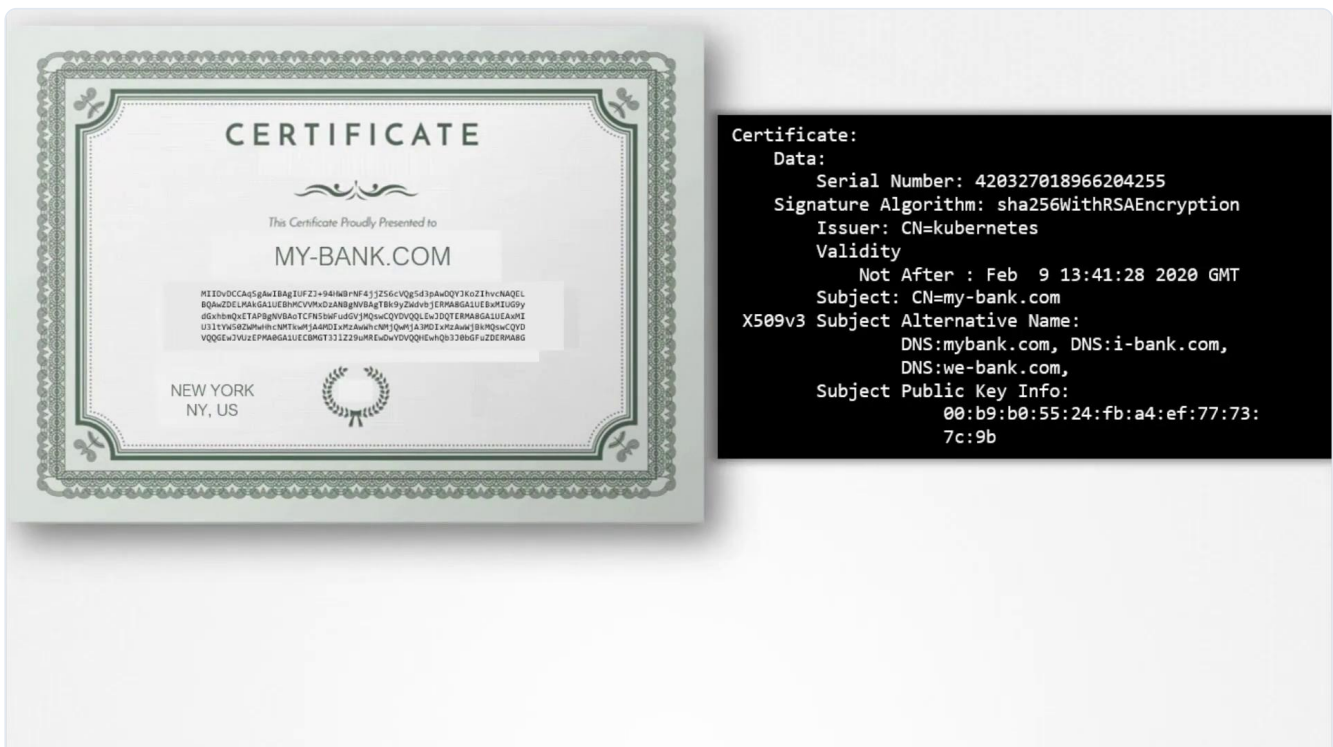
Imagine a hacker trying to intercept your bank communications by setting up a counterfeit website. The at

A certificate contains essential details that help verify its authenticity:

- * Identity of the issuing authority
- * The server's public key
- * Domain and other related information

Below is an example excerpt from a certificate:

```
```plaintext theme={null}
Certificate:
Data:
 Serial Number: 420327018966204255
 Signature Algorithm: sha256WithRSAEncryption
 Issuer: CN=kubernetes
 Validity
 Not After : Feb 9 13:41:28 2020 GMT
 Subject: CN=my-bank.com
 X509v3 Subject Alternative Name:
 DNS:mybank.com, DNS:i-bank.com,
 DNS:we-bank.com,
 Subject Public Key Info:
 00:b9:b0:55:24:fb:a4:ef:77:73:7c:9b
```



Browsers rely on Certificate Authorities (CAs) to sign and validate certificates. Renowned CAs, such as Symantec, DigiCert, Komodo, and GlobalSign, use their private keys to sign certificate signing requests (CSRs). When you generate a CSR for your web server, it is sent to a CA for signing:

```
bash theme={null} openssl req -new -key my-bank.key -out my-bank.csr -subj "/C=US/ST=CA/O=MyOrg, Inc./CN=my-bank.com"
```

Once your details are validated, the CA signs the certificate and sends it back to be installed on your web server. When a user accesses your website, the process is as follows:

1. The server presents the certificate.
2. The browser validates it using pre-installed CA public keys.
3. Upon successful validation, the browser and server establish a secure session using a symmetric key exchanged via asymmetric encryption.

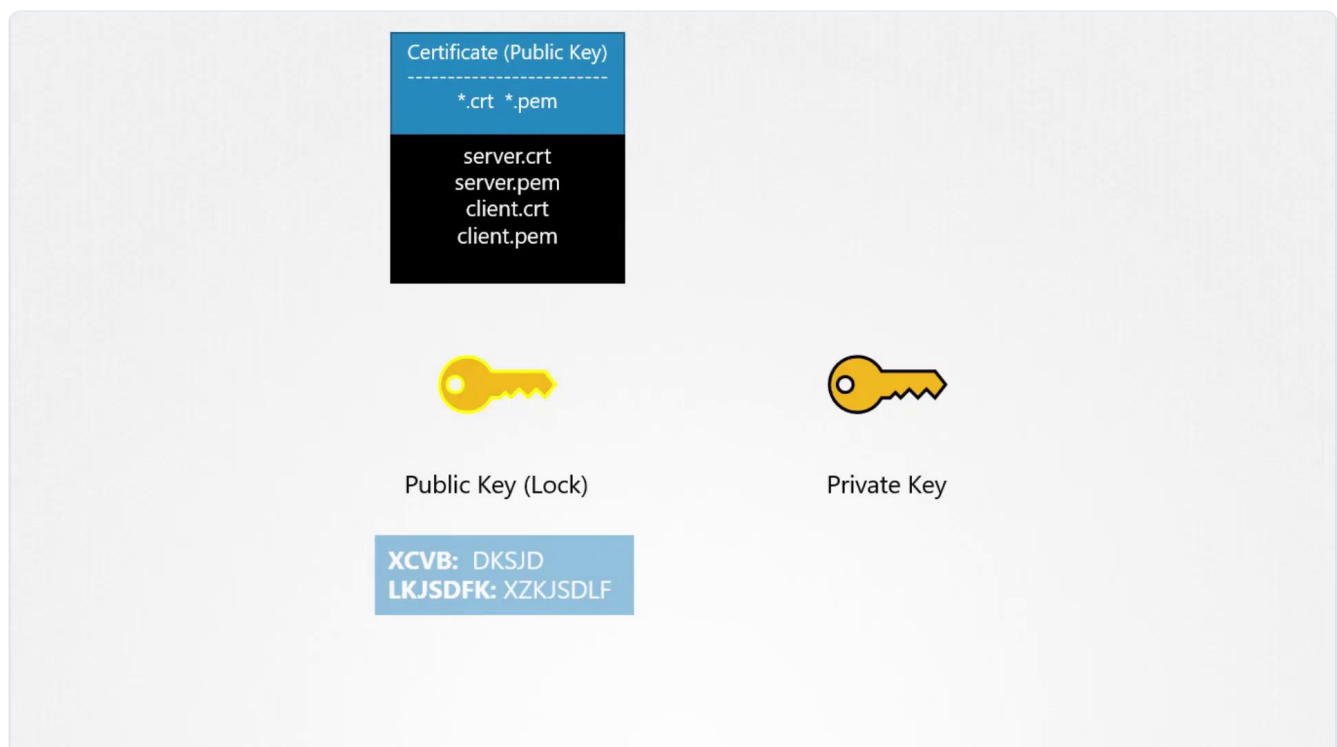
For internal systems, such as corporate payroll applications, organizations may deploy their own private CA and distribute its public key to employee devices.

#### :::note Key Points Summary

- Asymmetric encryption uses a pair of keys (public and private) to securely exchange symmetric keys.
- SSH access is secured using key pairs.
- Web servers use CA-signed certificates to establish HTTPS connections.
- A Certificate Signing Request (CSR) is generated and sent to a CA for signing.
- Signed certificates, combined with the server's key pair, secure the communication session. :::

It is important to note that although both keys in an asymmetric pair can encrypt data, only the complementary key can decrypt it. For instance, data encrypted with your private key can be decrypted by anyone with your public key; therefore, it's crucial to use the correct key for each operation.

Regarding file naming conventions, certificates containing public keys typically have extensions such as .crt or .pem (e.g., server.crt, server.pem or client.crt, client.pem), and private key files usually include "key" in the filename or extension (e.g., server.key or server-key.pem).



That concludes this lesson on TLS-Basics. Thank you for reading, and I look forward to seeing you in the next lesson!

## Additional Resources

---

- [Kubernetes Documentation](#)
- [SSL/TLS Basics](#)
- [Understanding Cryptography](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# TLS in Kubernetes

This article provides a guide on securing Kubernetes clusters using TLS certificates, covering key concepts, components, and management practices.

Welcome to this comprehensive guide on securing your Kubernetes cluster using TLS certificates. In this lesson, we cover the fundamentals of public and private keys, the role of certificate authorities (CA), and how both server and client certificates are employed to safeguard connections within your cluster.

Certificate files follow specific naming conventions. Public key certificates typically have a .crt or .pem extension (e.g., server.crt, server.pem, client.crt, client.pem). In contrast, private keys usually include the word "key" in their file name or extension (e.g., server.key or server-key.pem). If a file name lacks "key," it is almost certainly a public key certificate.

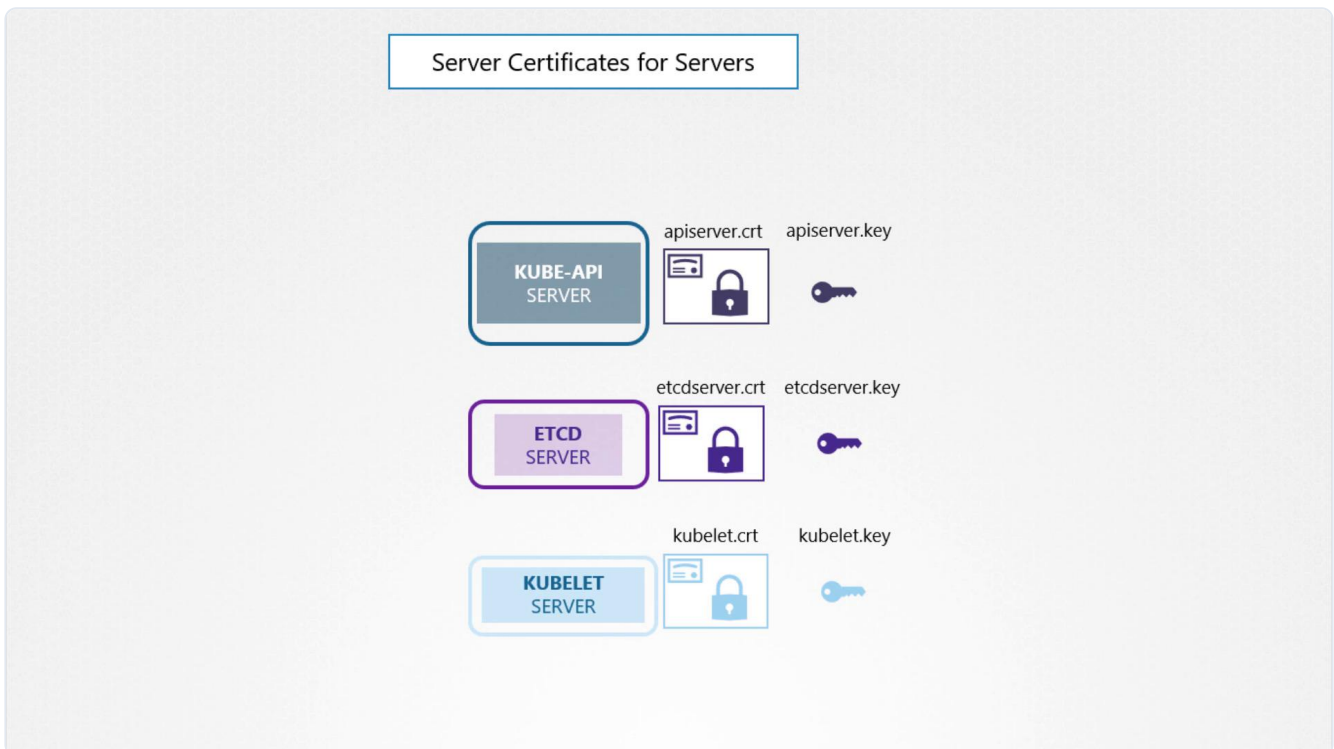
## TLS Certificates in the Kubernetes Cluster

A Kubernetes cluster consists of master and worker nodes that require secure, encrypted communication. Whether the connection is being made by an administrator using the kubectl utility or directly interacting with the Kubernetes API, a secure TLS connection is essential. Additionally, services within the cluster use server certificates to secure their communications, while client certificates authenticate users or other cluster components.

### Kubernetes Components and Their Certificates

Below is an overview of the main Kubernetes components and the associated certificate configurations:

- Kube API Server** \ The API server exposes an HTTPS service for managing the cluster. It acts as a server and requires its own certificate and key pair (e.g., `api-server.crt` and `api-server.key`). \ *Note: Certificate names may vary based on your cluster setup.*
- ETCD Server** \ Serving as the datastore for all cluster information, ETCD requires a unique pair of certificates and keys (e.g., `etcd-server.crt` and `etcd-server.key`).
- Kubelet on Worker Nodes** \ On each worker node, the kubelet service exposes an HTTPS endpoint that communicates with the API server for monitoring and management. This service uses its own certificate and key pair (typically named `kubelet.crt` and `kubelet.key`).

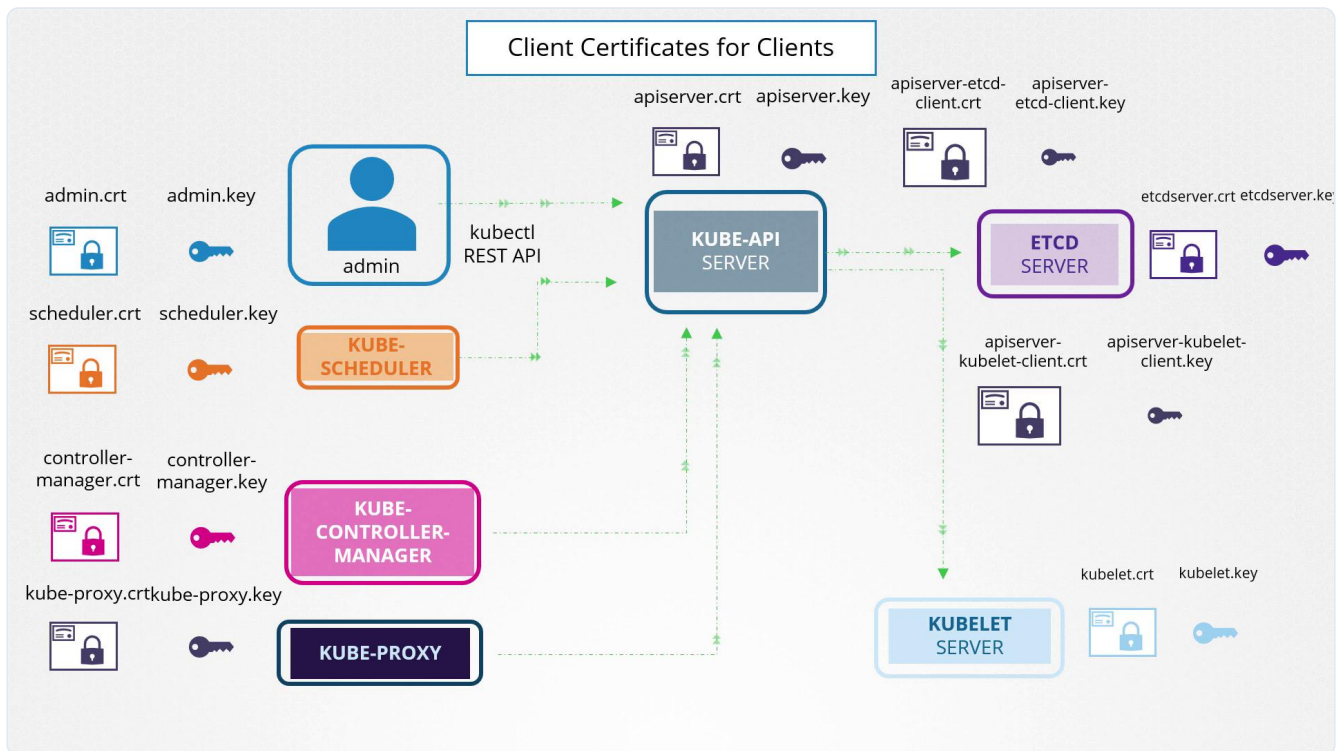


## Client Components and Their Certificates

Several components operate as clients and require authentication when connecting to the Kube API Server:

1. **Administrator (kubectl/REST API)** The admin user employs a client certificate and key pair (e.g., `admin.crt` and `admin.key`) to establish a secure connection to the API server.
2. **Scheduler** Managing pod scheduling, the scheduler requires its own client certificate (e.g., `scheduler.crt` and `scheduler.key`) to interact with the API server.
3. **Kube Controller Manager** This component also utilizes a dedicated certificate pair for authenticating its requests to the API server.
4. **Kube Proxy** Handling network routing within the cluster, the kube proxy uses a client certificate (generally named `kube-proxy.crt` and `kube-proxy.key`) for secure communications with the API server.

The Kube API Server facilitates secure communications with both the ETCD server and the kubelets. For instance, when connecting to the ETCD server, the API server can either reuse its HTTPS service certificates or use a dedicated certificate pair. Likewise, interactions with kubelets can leverage either the API server's default certificate pair or a separately generated set of certificates.



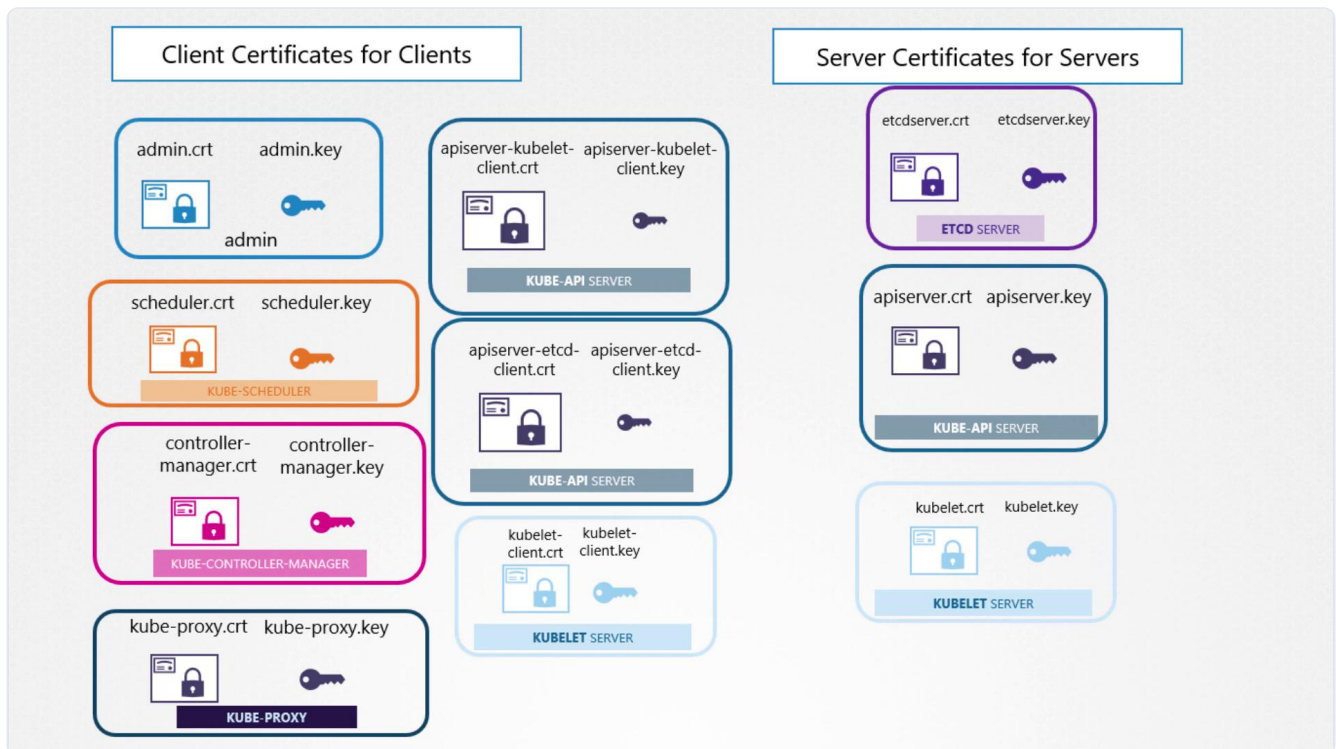
## Grouping Certificates for Enhanced Management

For easier certificate management, you can group TLS certificates into two main categories:

Category	Usage
Client Certificates	Used by components (e.g., admin, scheduler, controller manager, kube-proxy) to authenticate with the API server.
Server Certificates	Used by server components (e.g., Kube API Server, ETCD Server, kubelet) to secure their services.

## Utilizing a Certificate Authority (CA)

All certificates must be signed by a Certificate Authority. Kubernetes clusters require at least one CA, though multiple CAs may be used for additional segregation (for example, one CA for general cluster services and another exclusively for ETCD). In this guide, we use a single CA for simplicity. The CA itself has a certificate and key pair, typically named `ca.crt` and `ca.key`.



This overview on TLS certificates in Kubernetes sets the foundation for generating and managing these certificates to secure your cluster communications effectively. Happy securing!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# TLS in Kubernetes Certificate Creation

This guide explains how to generate TLS certificates for a Kubernetes cluster using OpenSSL, focusing on CA and client/server certificates.

In this guide, you will learn how to generate TLS certificates for a Kubernetes cluster using OpenSSL. While there are other popular tools such as EasyRSA and CFSSL, we will focus on using OpenSSL to create both Certificate Authority (CA) and client/server certificates. This approach secures the communications within your Kubernetes cluster and ensures certified access.

## 1. Generating CA Certificates

First, create the Certificate Authority (CA) certificates. The process involves generating a private key, creating a Certificate Signing Request (CSR) that includes the CA's common name, and finally signing the CSR with the private key to produce the CA certificate. The completed process provides the CA with its private key ( `ca.key` ) and root certificate ( `ca.crt` ), which are essential for subsequently signing other certificates.

```
```bash theme={null} openssl genrsa -out ca.key 2048 openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt
```

<Callout icon="lightbulb" color="#1CB2FE">

The CA certificate is the cornerstone of your certificate infrastructure, so ensure that you safeguard it.

</Callout>

2. Creating Client Certificates

2.1 Admin User Certificate

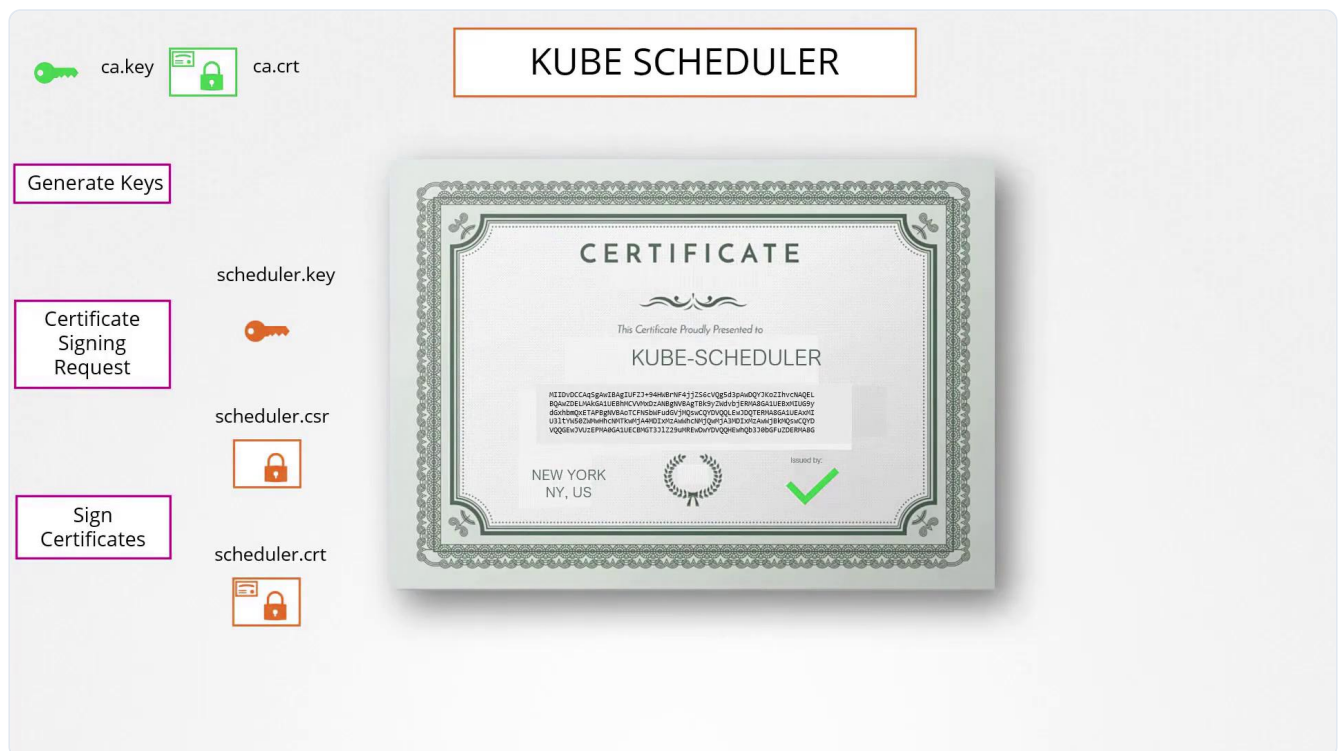
To generate a certificate for the admin user:

1. Create a private key for the admin.
2. Generate a CSR for the admin user specifying the common name (CN) and organizational unit (OU) to reflect the user's role.
3. Sign the admin CSR with the CA certificate to produce the final admin certificate.

```
```bash
theme={null}
openssl genrsa -out admin.key 2048
openssl req -new -key admin.key -subj "/CN=kube-admin/O=system:masters" -out admin.csr
openssl x509 -req -in admin.csr -CA ca.crt -CAkey ca.key -out admin.crt
```

The resulting `admin.crt` file functions as a secure credential, akin to a username and password pair, for authenticating the admin user with the Kubernetes API server.

A similar process is followed to generate client certificates for other components such as the scheduler, controller manager, and kube-proxy.



### 3. Using Client Certificates in API Requests

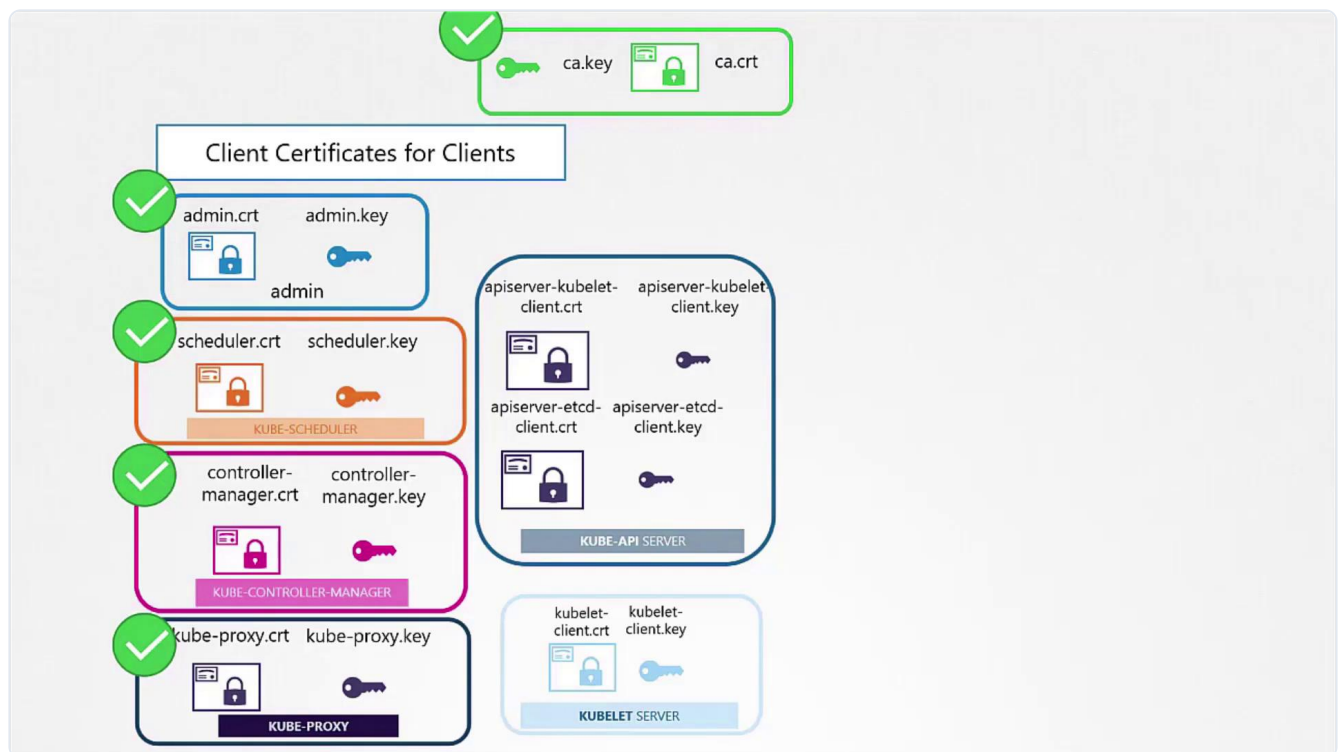
Client certificates eliminate the requirement for using a username and password when making REST API calls to the Kubernetes API server. The admin certificate, for example, can be used to securely communicate with the server by specifying the key, certificate, and CA certificate in your request.

```
```bash theme={null} curl https://kube-apiserver:6443/api/v1/pods \ --key admin.key --cert admin.crt --cacert ca.crt
```

The API server will respond with a JSON object listing the pods:

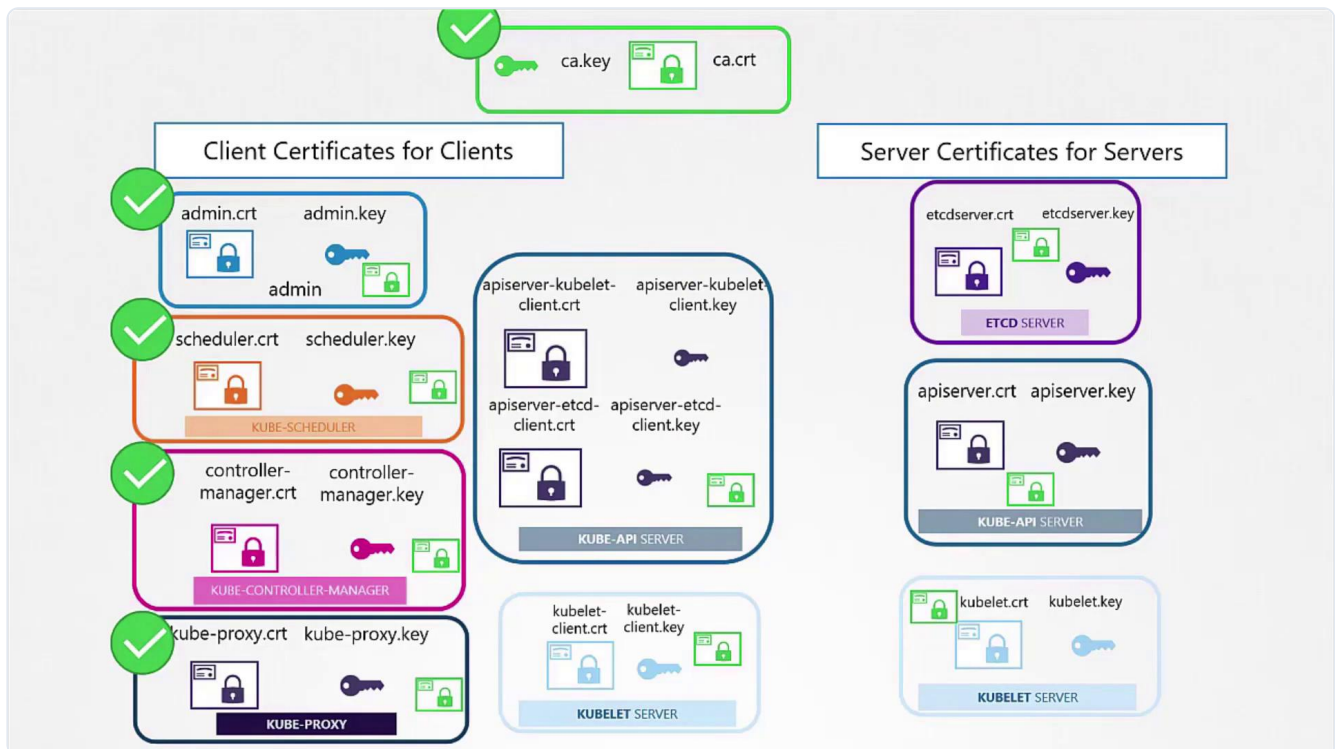
```
```json theme={null}
{
 "kind": "PodList",
 "apiVersion": "v1",
 "metadata": {
 "selfLink": "/api/v1/pods"
 },
 "items": []
}
```

Most Kubernetes clients can load these connection parameters via a kubeconfig file that consolidates the information required to reach the API server.



### 4. Server-Side Certificates

For secure communication, both client and server certificates must trust the same CA root certificate. This certificate is used by both parties to verify the authenticity of the certificate they receive.



#### 4.1 Etcd Server Certificate

The etcd server, a critical component in high availability configurations, also requires a certificate. If etcd is running as a cluster, remember to generate peer certificates to secure inter-member communications. Once created, the certificates are referenced in the etcd configuration file (commonly, `etcd.yaml`). See the example below:

```
```bash theme={null} cat etcd.yaml - --advertise-client-urls=https://127.0.0.1:2379 - --key-file=/path-to-certs/etcdserver.key - --cert-file=/path-to-certs/etcdserver.crt - --client-cert-auth=true - --data-dir=/var/lib/etcd - --initial-advertise-peer-urls=https://127.0.0.1:2380 - --initial-cluster=master=https://127.0.0.1:2380 - --listen-client-urls=https://127.0.0.1:2379 - --listen-peer-urls=https://127.0.0.1:2380 - --name=master - --peer-cert-file=/path-to-certs/etcdpeer1.crt - --peer-client-cert-auth=true - --peer-key-file=/etc/kubernetes/pki/etcd/peer.key - --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt - --snapshot-count=10000 - --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

The `--trusted-ca-file` option ensures that etcd client connections are authenticated using the CA certificate.

5. Kube API Server Certificates

The Kube API server is the primary access point for the cluster and is known by several aliases such as "kubernetes"

5.1 Creating the API Server Certificate

Start by generating a CSR for the API server:

```
```bash theme={null}
openssl req -new -key apiserver.key -subj "/CN=kube-apiserver" -out apiserver.csr
```

Then, create an OpenSSL configuration file (e.g., `openssl.cnf`) to include all necessary SANs:

```

[req] req_extensions = v3_req distinguished_name = req_distinguished_name

[v3_req] basicConstraints = CA:FALSE keyUsage = nonRepudiation, digitalSignature, keyEncipherment
subjectAltName = @alt_names

[alt_names] DNS.1 = kubernetes DNS.2 = kubernetes.default DNS.3 = kubernetes.default.svc DNS.4 =
kubernetes.default.svc.cluster.local IP.1 = 10.96.0.1 IP.2 = 172.17.0.87

```

After configuring the CSR with the SANs, sign the certificate using your CA certificate and key. Specify

```

```bash theme={null}
ExecStart=/usr/local/bin/kube-apiserver \
  --advertise-address=${INTERNAL_IP} \
  --allow-privileged=true \
  --apiserver-count=3 \
  --authorization-mode=Node,RBAC \
  --bind-address=0.0.0.0 \
  --enable-swagger-ui=true \
  --etcd-cafile=/var/lib/kubernetes/ca.pem \
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \
  --etcd-servers=https://127.0.0.1:2379 \
  --event-ttl=1h \
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \
  --kubelet-client-certificate=/var/lib/kubernetes/apiserver-kubelet-client.crt \
  --kubelet-client-key=/var/lib/kubernetes/apiserver-kubelet-client.key \
  --kubelet-https=true \
  --runtime-config=api/all \
  --service-account-key-file=/var/lib/kubernetes/service-account.pem \
  --service-cluster-ip-range=10.32.0.0/24 \
  --service-node-port-range=30000-32767 \
  --client-ca-file=/var/lib/kubernetes/ca.pem \
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \
  --v=2

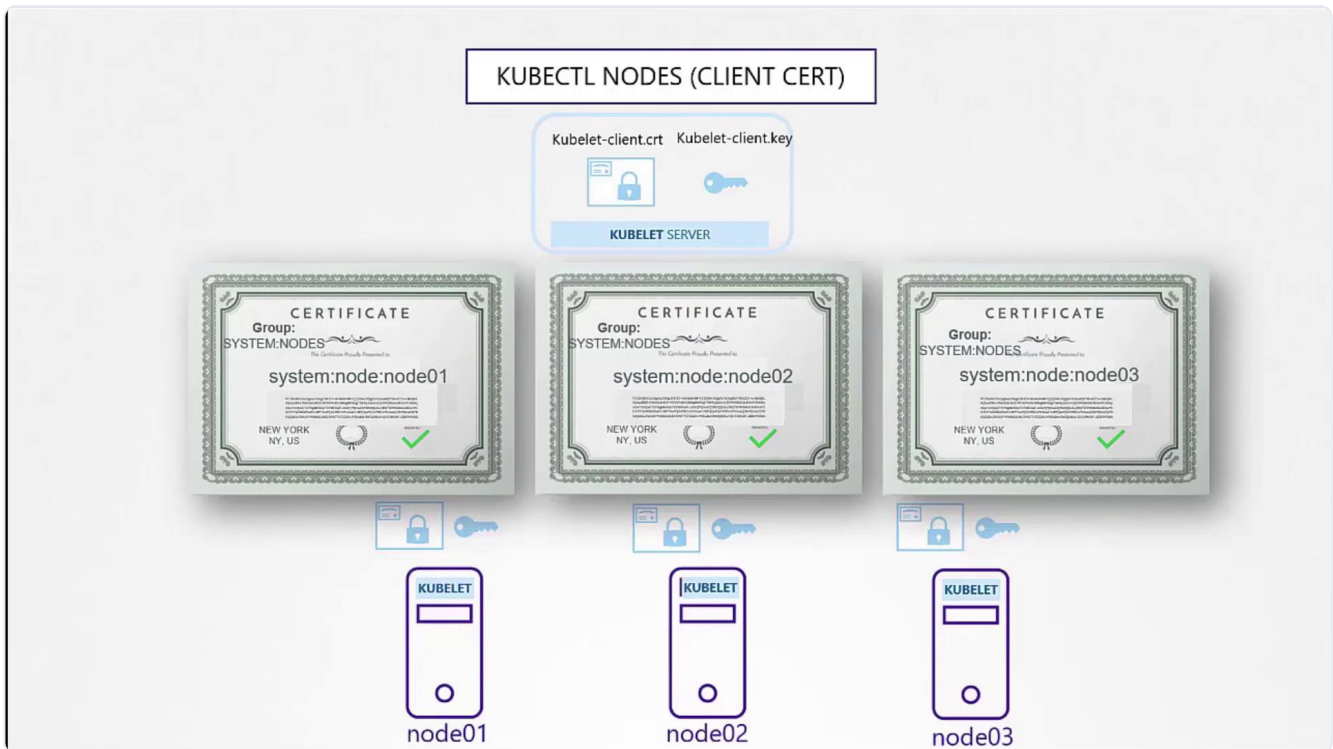
```

Ensure that all DNS names and IP addresses used by the API server are correctly listed in the SAN section of your OpenSSL configuration.

6. Kubelet Certificates

The kubelet is a critical component running on each node, managing node-specific operations and secure communication with the API server. For this purpose, every node needs its own certificate and key pair. When generating these certificates, name them according to the node's identity (e.g., `node01`, `node02`, `node03`).

It is also a best practice to generate a separate certificate for the node when it acts as a client to the API server. This certificate should include an identity format such as `"system:node"` to ensure the API server can assign the appropriate group membership (e.g., `system:nodes`).



Once generated, include these certificates in the kubeconfig files for the respective nodes to guarantee secure communication.

Conclusion

This guide has detailed the steps required to generate both client and server certificates using OpenSSL in a Kubernetes environment. You learned how to create a CA certificate, generate and sign client certificates (including for administrative use), and produce server certificates for both etcd and the API server. By including multiple domain names and IP addresses in your certificates, you ensure a robust and flexible security architecture for your cluster.

Future topics will explore how to inspect certificate details and how tools like kubeadm streamline certificate management in Kubernetes clusters.

Additional Resources

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

View Certificate Details

Learn to inspect and verify certificates in a Kubernetes cluster, covering both manual setups and automated configurations like kubeadm.

Hello, and welcome to this lesson. In this guide, you will learn how to inspect and verify certificates in an existing Kubernetes cluster. Imagine joining a new team responsible for a Kubernetes environment where you need to perform a health check on cluster certificates due to reported issues. This lesson covers both manually set-up clusters and those configured using automated tools like kubeadm.

Understanding Your Cluster Setup

Before diving into certificate verification, it's important to understand your cluster's setup. There are several methods for deploying a Kubernetes cluster, and each has its own approach to generating and managing certificates.

- If you deploy a cluster from scratch, you may generate and configure all certificates manually (as explored in a previous lesson).
- If you use an automated provisioning tool like kubeadm, certificate generation and configuration are handled for you. In this case, Kubernetes components are deployed as pods instead of OS services.

Native Service Deployment

When Kubernetes components are deployed as native services, you can review service files to understand the certificate configuration. For example, inspect the kube-apiserver service file:

```
```bash theme={null} cat /etc/systemd/system/kube-apiserver.service [Service] ExecStart=/usr/local/bin/kube-apiserver \ --advertise-address=172.17.0.32 \ --allow-privileged=true \ --apiserver-count=3 \ --authorization-mode=Node,RBAC \ --bind-address=0.0.0.0 \ --client-ca-file=/var/lib/kubernetes/ca.pem \ --enable-swagger-ui=true \ --etcd-cafile=/var/lib/kubernetes/ca.pem \ --etcd-certfile=/var/lib/kubernetes/kubernetes.pem \ --etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \ --event-ttl=1h \ --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \ --kubelet-client-certfile=/var/lib/kubernetes/kubelet-client.crt \ --kubelet-client-key=/var/lib/kubernetes/kubelet-client.key \ --kubelet-https=true \ --service-node-port-range=30000-32767 \ --tls-cert-file=/var/lib/kubernetes/kube-apiserver.crt \ --tls-private-key-file=/var/lib/kubernetes/kube-apiserver-key.pem \ --v=2
```

### ### Deployment Using kubeadm

When using kubeadm, components such as the kube-apiserver are defined as pods in manifest files. For example,

```
```yaml theme={null}
cat /etc/kubernetes/manifests/kube-apiserver.yaml
spec:
  containers:
    - command:
      - kube-apiserver
      - --authorization-mode=Node,RBAC
      - --advertise-address=172.17.0.32
      - --allow-privileged=true
      - --client-ca-file=/etc/kubernetes/pki/ca.crt
      - --disable-admission-plugins=PersistentVolumeLabel
      - --enable-admission-plugins=NodeRestriction
      - --enable-bootstrap-token-auth=true
      - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
      - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
      - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
      - --insecure-port=0
      - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
      - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
      - --proxy-client-certfile=/etc/kubernetes/pki/apiserver-kubelet-client.crt
      - --proxy-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
      - --request-timeout=30s
```

Creating a Certificate Inventory

When performing a certificate health check, it's essential to create a checklist—perhaps using a spreadsheet—to record details such as:

- Certificate file paths
- Configured names and alternate names
- Associated organizations
- Certificate owners
- Certificate authorities (issuers)
- Expiration dates

Begin by examining configuration files (such as the kube-apiserver manifest located in `/etc/kubernetes/manifests`) to identify the certificate files in use.

For example, the kube-apiserver manifest might reveal the following options:

```
```yaml theme={null} spec: containers: - command: - kube-apiserver - --authorization-mode=Node,RBAC - --
advertise-address=172.17.0.32 - --allow-privileged=true - --client-ca-file=/etc/kubernetes/pki/ca.crt - --disable-
admission-plugins=PersistentVolumeLabel - --enable-admission-plugins=NodeRestriction - --enable-bootstrap-
token-auth=true - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-
client.crt - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key - --etcd-servers=https://127.0.0.1:2379 - --
insecure-port=0 - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt - --kubelet-client-
```

```
key=/etc/kubernetes/pki/apiserver-kubelet-client.key - --kubelet-preferred-address-
types=InternalIP,ExternalIP,Hostname - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt - --proxy-
client-key-file=/etc/kubernetes/pki/front-proxy-client.key - --secure-port=6443 - --service-account-key-file=/etc/
kubernetes/pki/sa.pub - --service-cluster-ip-range=10.96.0.0/12 - --tls-cert-file=/etc/kubernetes/pki/apiserver.crt -
--tls-private-key-file=/etc/kubernetes/pki/apiserver.key
```

<Callout icon="lightbulb" color="#1CB2FE">

Make sure to document all certificate details as part of your checklist. This practice simplifies troubleshooting.

</Callout>

### ## Inspecting Certificate Details

After identifying certificate files, use OpenSSL to decode them and check their details. For example, to inspect the

```
```bash theme={null}
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout
```

This command displays:

- The subject name and any alternate names
- The validity period (including expiry dates)
- The issuing certificate authority

Repeat this process for all certificates in your Kubernetes cluster. Ensure that:

- Certificate names and alternate names are correctly configured.
- Each certificate is associated with the appropriate organization.
- Certificates are issued by the correct certificate authority (e.g., kubeadm typically uses "Kubernetes" as the CA).
- None of the certificates have expired.

An expired or misconfigured certificate can cause critical disruptions in your cluster's operations. Always validate certificate expiry dates and issuer details.

Troubleshooting with Logs

When certificate issues are suspected, reviewing logs can provide valuable insights.

For Clusters Using Native OS Services

Check service logs using system commands. For example, inspect etcd logs with:

```
```bash theme={null} journalctl -u etcd.service -l
```

Below is an example excerpt from etcd logs:

```
``plaintext theme={null}
2019-02-13 02:53:28.144631 I | etcdmain: etcd Version: 3.2.18
2019-02-13 02:53:28.144680 I | etcdmain: Git SHA: eddf599c6
2019-02-13 02:53:28.144684 I | etcdmain: Go Version: go1.8.7
2019-02-13 02:53:28.144692 I | etcdmain: Go OS/Arch: linux/amd64
2019-02-13 02:53:28.144696 I | etcdmain: setting maximum number of CPUs to 4, total number of available CPUs is 4
2019-02-13 02:53:28.144734 N | etcdmain: the server is already initialized as member before, starting as etcdserver
2019-02-13 02:53:28.146651 I | etcdserver: name = master
...
WARNING: 2019/02/13 02:53:30 Failed to serve client requests on 127.0.0.1:2379
Failed to dial 127.0.0.1:2379: connection error: desc = "transport: authentication handshake failed: remote error: http2: stream error: stream ID 1: EOF"
```

## For Clusters Using kubeadm

Since core components are deployed as pods, retrieve logs using:

- Running `kubectl logs <pod-name>` for pod-level logs.
- If the API server or etcd is down and `kubectl` is unresponsive, list all containers with:

```
bash theme={null} docker ps -a
```

Then inspect container logs:

```
bash theme={null} docker logs <container-id>
```

## Additional Resources

For further details on certificate requirements and best practices, please refer to the official [Kubernetes Documentation](#) and explore additional resources available on the Kubernetes website.

That's it for this lesson. Head over to your practice environment and gain hands-on experience in viewing and verifying certificates in an existing Kubernetes cluster.

Default CN	Parent CA	O (in Subject)	kind	hosts (SAN)
kube-etcd	etcd-ca		server, client [1]etcdbug]	localhost, 127.0.0.1
kube-etcd-peer	etcd-ca		server, client	<hostname>, <Host_IP>, localhost, 127.0.0.1
kube-etcd-healthcheck-client	etcd-ca		client	
kube-apiserver-etcd-client	etcd-ca	system:masters	client	
kube-apiserver	kubernetes-ca		server	<hostname>, <Host_IP>, <advertise_IP>, [1]
kube-apiserver-kubelet-client	kubernetes-ca	system:masters	client	
front-proxy-client	kubernetes-front-proxy-ca		client	

Default CN	recommended key path	recommended cert path	command	key argument	cert argument
etcd-ca		etcd/ca.crt	kube-apiserver		-etcd-cafile
etcd-client	apiserver-etcd-client.key	apiserver-etcd-client.crt	kube-apiserver	-etcd-keyfile	-etcd-certfile
kubernetes-ca		ca.crt	kube-apiserver		-client-ca-file
kube-apiserver	apiserver.key	apiserver.crt	kube-apiserver	-tls-private-key-file	-tls-cert-file
apiserver-kubelet-client		apiserver-kubelet-client.crt	kube-apiserver		-kubelet-client-certificate
front-proxy-ca		front-proxy-ca.crt	kube-apiserver		-requestheader-client-ca-file
front-proxy-client	front-proxy-client.key	front-proxy-client.crt	kube-apiserver	-proxy-client-key-file	-proxy-client-cert-file
etcd-ca		etcd/ca.crt	etcd		-trusted-ca-file, -peer-trusted-ca-file
kube-etcd	etcd/server.key	etcd/server.crt	etcd	-key-file	-cert-file
kube-etcd-peer	etcd/peer.key	etcd/peer.crt	etcd	-peer-key-file	-peer-cert-file
etcd-ca		etcd/ca.crt	etcdctl[2]		-cacert
kube-etcd-healthcheck-client	etcd/healthcheck-client.key	etcd/healthcheck-client.crt	etcdctl[2]	-key	-cert

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Certificates API

This article explains managing certificates and the Certificate API in Kubernetes, detailing the lifecycle of certificate signing requests and automation for certificate rotation.

This article explains how to manage certificates and introduces the Certificate API in Kubernetes. In a typical cluster setup, an administrator first configures a Certificate Authority (CA) server and generates certificates for the various components. Once the services are launched with these certificates, the cluster becomes operational. Initially, only one administrator has access via a personal admin certificate and key. However, as new team members join, each must obtain their own certificate and key pair to access the cluster.

## The Certificate Lifecycle

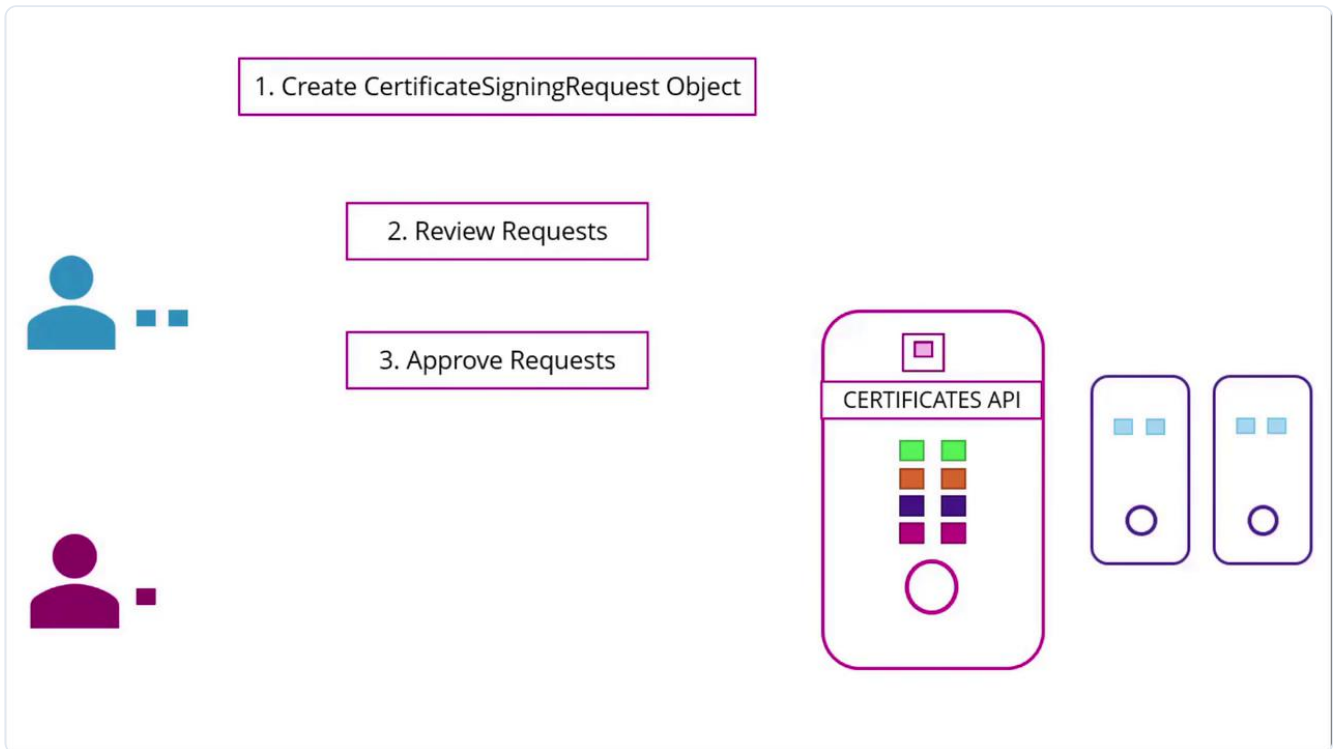
When a new administrator joins, she generates her own private key and creates a certificate signing request (CSR). This CSR is then sent to the existing administrator. As the sole administrator, you review the CSR, sign it using the CA server's private key and root certificate, and return the signed certificate. The new admin now has a valid certificate and key pair to access the cluster. Since certificates have a defined expiration period, the process is repeated when they expire.

The Kubernetes API Server plays a pivotal role in the cluster but is not part of the Certificate Authority. The CA is comprised of only two files—a key and a certificate—generated during initialization. Because these files allow the signing of certificates and thus the creation of users with any privileges, they must be stored securely, typically on a dedicated CA server. In many implementations, the Kubernetes master node also serves as the CA server. For instance, the kubeadm tool creates and stores CA files on the master node.

As the number of users increases, manually signing certificate requests becomes impractical. Kubernetes addresses this challenge with a built-in Certificates API that automates CSR management and certificate rotation.

## Managing Certificate Signing Requests (CSRs)

The Kubernetes Certificates API allows users to submit their CSRs via an API call, creating a CertificateSigningRequest object. Administrators can then review and approve these requests using `kubectl` commands. Once approved, Kubernetes signs the certificate using the CA's key pair. The signed certificate is then available for extraction and distribution to the requesting user.



### Step 1: User Generates Private Key and CSR

A user creates a private key and generates a certificate signing request using the following command:

```
```bash theme={null} openssl genrsa -out jane.key 2048
```

The user then sends the CSR to the administrator.

Step 2: Administrator Creates a CSR Object

The administrator creates a CertificateSigningRequest object with a manifest file. In the manifest, the

```
```yaml theme={null}
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
 name: jane
spec:
 expirationSeconds: 600 # seconds
 usages:
 - digital signature
 - key encipherment
 - server auth
 request: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBSRVFVRVNUUw0tLS0KTUl1Q1dEQ0NBVUFdQVFBd0V6RVJHQTFFVdU0R6
```

Administrators can list pending CSRs with the following command:

```
```bash theme={null} kubectl get csr
```

The output may resemble:

```
```plaintext theme={null}
NAME AGE SIGNERNAME REQUESTOR REQUESTEDDURATION
jane 10m kubernetes.io/kube-apiserver-client admin@example.com 10m
```

### Step 3: Approving the CSR

To approve the CSR, run:

```
```bash theme={null} kubectl certificate approve jane
```

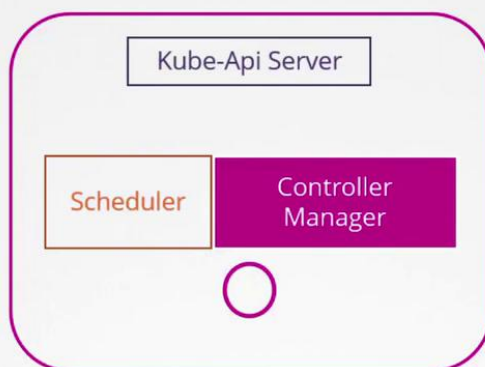
After approval, Kubernetes signs the CSR with the CA key pair, and the certificate is embedded in the CertificateSigningRequest object.

Below is an example output of a CertificateSigningRequest object:

```
```yaml theme={null}
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
 creationTimestamp: 2019-02-13T16:36:43Z
 name: new-user
spec:
 groups:
 - system:masters
 - system:authenticated
 expirationSeconds: 600
 usages:
 - digital signature
 - key encipherment
 - server auth
 username: kubernetes-admin
status:
 certificate: LS0tL1CRUdJTlBDRVJUSUZJQ0FURS9tL0t1SURDakNDQWZLZ0F3SUJBZ0lVRmwyQ2wXYXoxalw5M3JNVissFRYQY0
 conditions:
 - lastUpdateTime: 2019-02-13T16:37:21Z
 message: This CSR was approved by kubectl certificate approve.
 reason: KubectlApprove
 type: Approved
```

## The Role of the Controller Manager

Within the Kubernetes control plane, components such as the API Server, Scheduler, and Controller Manager work together. However, all certificate-related operations—such as CSR approval and signing—are managed by the Controller Manager.



The Controller Manager includes dedicated controllers for CSR approval and CSR signing tasks. Since signing certificates requires access to the CA's root certificate and private key, its configuration specifies the file paths to these credentials. For example, the Controller Manager's configuration file might include settings like the following:

```
yaml theme={null} cat /etc/kubernetes/manifests/kube-controller-manager.yaml spec: containers: -
command: - kube-controller-manager - --address=127.0.0.1 - --cluster-signing-cert-file=/etc/
kubernetes/pki/ca.crt - --cluster-signing-key-file=/etc/kubernetes/pki/ca.key - --
```

```
controllers=*,bootstrapsigner,tokencleaner - --kubeconfig=/etc/kubernetes/controller-manager.conf
- --leader-elect=true - --root-ca-file=/etc/kubernetes/pki/ca.crt - --service-account-private-
key-file=/etc/kubernetes/pki/sa.key - --use-service-account-credentials=true
```

This configuration demonstrates how the Controller Manager accesses the necessary CA certificates for signing new certificate requests.

## Conclusion

---

In summary, Kubernetes simplifies certificate management through its built-in Certificates API, automating the CSR lifecycle and certificate rotation. This automation is essential for maintaining secure access as team sizes grow and certificates expire over time.

For further exploration, head over to the practice desk to experiment with the Certificates API and enhance your Kubernetes operations.

See you in the next article!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# KubeConfig

This article explains kubeconfig files in Kubernetes, focusing on certificate-based authentication and access management across multiple clusters.

Welcome to this lesson on kubeconfig files in Kubernetes. In this session, we will dive into certificate-based authentication using both curl and kubectl, and demonstrate how a kubeconfig file simplifies access management across multiple clusters.

## Certificate Authentication with curl and kubectl

Previously, we generated a certificate for a user and utilized the certificate along with a key to query the Kubernetes REST API for a list of pods. For instance, if your cluster is named "my kube playground," you can make a curl request to the API server as follows:

```
```bash theme={null} curl https://my-kube-playground:6443/api/v1/pods \ --key admin.key \ --cert admin.crt \ --cacert ca.crt
```

The API server then returns a response similar to this:

```
```json theme={null}
{
 "kind": "PodList",
 "apiVersion": "v1",
 "metadata": {
 "selfLink": "/api/v1/pods"
 },
 "items": []
}
```

Likewise, when using the kubectl command-line tool, you can supply the same parameters:

```
```bash theme={null} kubectl get pods \ --server https://my-kube-playground:6443 \ --client-key admin.key \ --client-certificate admin.crt \ --certificate-authority ca.crt
```

The response in this case might be:

```
```bash theme={null}
No resources found.
```

Instead of typing these options each time, streamline your workflow by moving them into a kubeconfig file.

## Understanding the Kubeconfig File

By default, `kubectl` searches for a kubeconfig file named "config" in the `~/.kube` directory. Once properly set up, you can simply execute:

```
```bash theme={null} kubectl get pods
```

and `kubectl` will automatically use the configurations defined within the file.

The kubeconfig file is organized into three key sections:

- * **Clusters:** Define the Kubernetes clusters you need access to (e.g., development, production, or test).
- * **Users:** Specify the user accounts and associated credentials (such as admin, dev, or prod users) that can access the clusters.
- * **Contexts:** Link a cluster with a user by specifying which user should access which cluster. A context is a combination of a cluster and a user.

Below is an example of a basic kubeconfig file in YAML format:

```
```yaml theme={null}
apiVersion: v1
kind: Config
clusters:
- name: my-kube-playground # values hidden...
- name: development
- name: production
- name: google
contexts:
- name: my-kube-admin@my-kube-playground
- name: dev-user@google
- name: prod-user@production
users:
- name: my-kube-admin
- name: admin
- name: dev-user
- name: prod-user
```

In this configuration, the server specification for the "my kube playground" cluster is defined in the clusters section, the admin user's credentials are listed in the users section, and the context named `my-kube-admin@my-kube-playground` ties them together. Multiple contexts can be created for different clusters and users, and you can set a default context using the `current-context` field.

## Viewing and Customizing Your Kubeconfig

To view the current kubeconfig settings, run:

```
```bash theme={null} kubectl config view
```

This command outputs details about clusters, users, contexts, and the active context. An example output may look like the following:

```
```yaml theme={null}
apiVersion: v1
kind: Config
current-context: kubernetes-admin@kubernetes
clusters:
- cluster:
 certificate-authority-data: REDACTED
 server: https://172.17.0.5:6443
 name: kubernetes
contexts:
- context:
 cluster: kubernetes
 user: kubernetes-admin
 name: kubernetes-admin@kubernetes
users:
- name: kubernetes-admin
 user:
 client-certificate-data: REDACTED
 client-key-data: REDACTED
```

If you want to view a custom kubeconfig file, use the `--kubeconfig` option:

```
```bash theme={null} kubectl config view --kubeconfig=my-custom-config
```

A sample custom configuration may appear as follows:

```
```yaml theme={null}
apiVersion: v1
kind: Config
current-context: my-kube-admin@my-kube-playground
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

To change the active context—for example, switching from the admin user to the production user—execute:

```
```bash theme={null} kubectl config use-context prod-user@production
```

After running this command, the kubeconfig is updated accordingly. The new configuration might look like:

```
```yaml theme={null}
apiVersion: v1
kind: Config
current-context: prod-user@production
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

Additional kubectl config commands can be used to update or delete entries as needed.

## Configuring Default Namespaces

Namespaces in Kubernetes help segment clusters into multiple virtual clusters. You can configure a context to automatically use a specific namespace. Consider the following kubeconfig snippet without a default namespace:

```
```yaml theme={null} apiVersion: v1 kind: Config clusters: - name: production cluster: certificate-authority: ca.crt
server: https://172.17.0.51:6443 contexts: - name: admin@production context: cluster: production user: admin
users: - name: admin user: client-certificate: admin.crt client-key: admin.key
```

To specify a default namespace (for example, "finance"), simply add the `namespace` field:

```
```yaml theme={null}
apiVersion: v1
kind: Config
clusters:
- name: production
 cluster:
 certificate-authority: ca.crt
 server: https://172.17.0.51:6443
contexts:
- name: admin@production
 context:
 cluster: production
 user: admin
 namespace: finance
users:
- name: admin
 user:
 client-certificate: admin.crt
 client-key: admin.key
```

When you switch to this context, kubectl will automatically operate within the specified namespace.

## Managing Certificates in Kubeconfig Files

For best practices, use full paths for certificate files in your kubeconfig file. Alternatively, you can embed the certificate data directly using the `certificate-authority-data` field.

For instance, specifying a full path looks like this:

```
```yaml theme={null} apiVersion: v1 kind: Config clusters: - name: production cluster: certificate-authority: /etc/kubernetes/pki/ca.crt
```

Alternatively, you may embed the certificate data directly:

```
```yaml theme={null}
apiVersion: v1
kind: Config
clusters:
- name: production
 cluster:
 certificate-authority: /etc/kubernetes/pki/ca.crt
 certificate-authority-data: LS0tLS1CRUdJTiBD...
```

To decode base64 encoded certificate data, use the following command:

```
```bash theme={null} echo "LS0tLS1CRUdJTiBD..." | base64 --decode
```

The decoded output will resemble:

```
```text theme={null}
-----BEGIN CERTIFICATE-----
MIICDCCAUCAQAwE...
-----END CERTIFICATE-----
```

## Conclusion

This concludes our detailed exploration of kubeconfig files in Kubernetes. Use these best practices and examples to manage your clusters efficiently and troubleshoot any configuration issues you may encounter.

For further learning, explore the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Transcribed by <https://otter.ai>

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# API Groups

This article provides an in-depth look into Kubernetes API groups, their structure, and methods for querying the API server.

Before diving into authorization, it's essential to understand the concept of API groups in Kubernetes and how they integrate with overall cluster operations. This article provides an in-depth look into Kubernetes API groups, their structure, and the methods for querying the API server.

## Understanding the Kubernetes API

The Kubernetes API is the primary interface for interacting with your cluster. Whether using the command-line tool `kubectl` or directly sending HTTP requests via REST, every interaction communicates with the API server. For example, to check your cluster's version, run:

```
```bash theme={null} curl https://kube-master:6443/version
```

The response may look like:

```
```json theme={null}
{
 "major": "1",
 "minor": "13",
 "gitVersion": "v1.13.0",
 "gitCommit": "ddf47ac13c1a9483ea035a79cd7c1005ff21a6d",
 "gitTreeState": "clean",
 "buildDate": "2018-12-03T20:56:12Z",
 "goVersion": "go1.11.2",
 "compiler": "gc",
 "platform": "linux/amd64"
}
```

Likewise, listing pods in the cluster involves accessing the `/api/v1/pods` endpoint.

## API Groups and Their Purpose

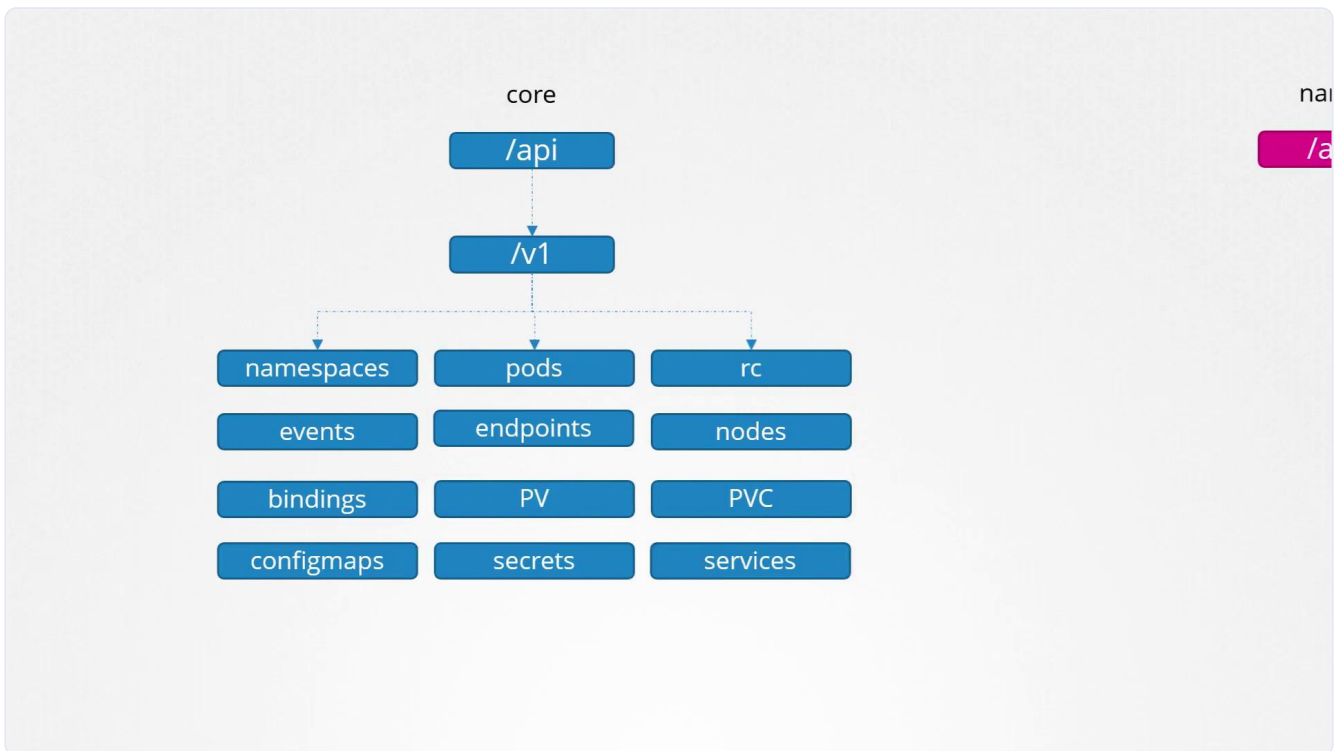
Kubernetes organizes its API into multiple groups based on specific functionality. These groups help in managing versioning, health metrics, logging, and more. For instance, the `/version` endpoint provides cluster version data, while endpoints like `/metrics` and `/healthz` offer insights into the cluster's performance and health.



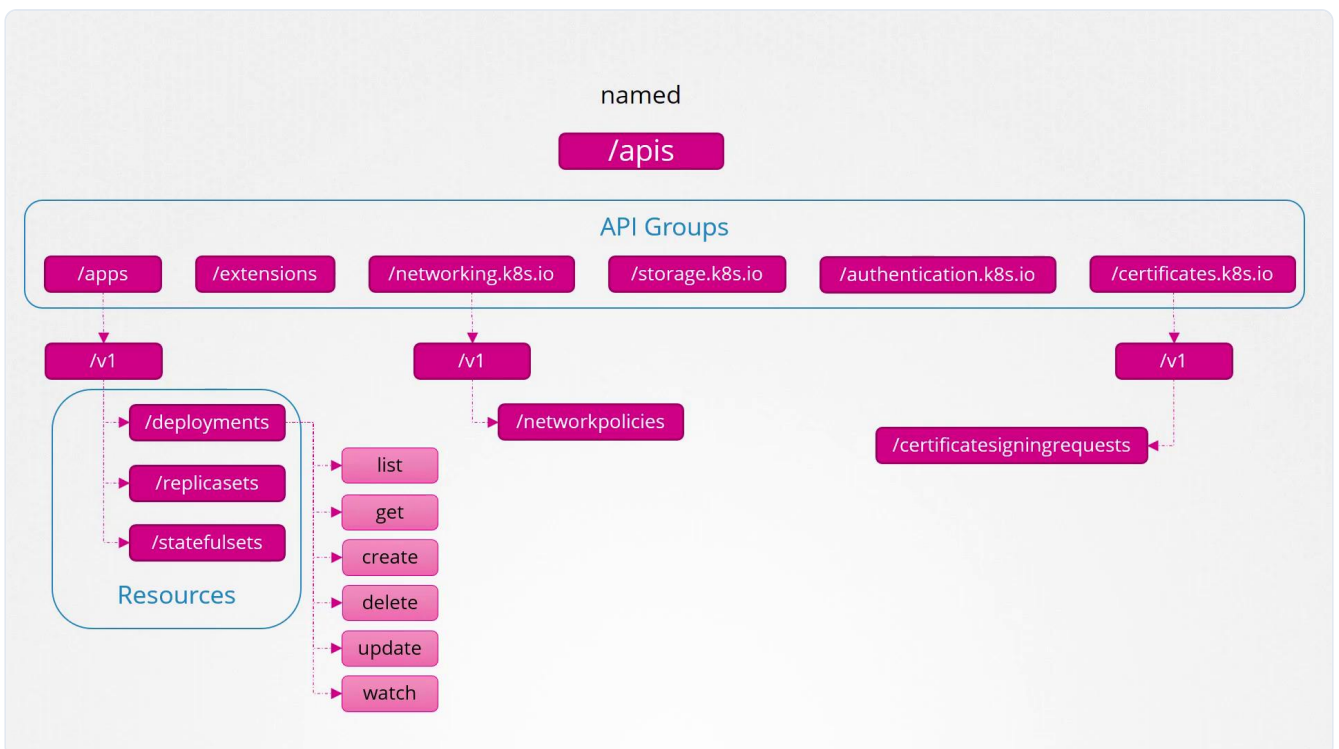
`/metrics` `/healthz` `/version` `/api` `/apis` `/logs`

This article focuses on two main API group categories:

1. **Core API Group:** Contains the essential features of Kubernetes such as namespaces, pods, replication controllers, events, endpoints, nodes, bindings, persistent volumes, persistent volume claims, config maps, secrets, and services.
2. **Named API Groups:** Provides an organized structure for newer features. These groups include apps, extensions, networking, storage, authentication, and authorization. For example, under the apps group, you'll find Deployments, ReplicaSets, and StatefulSets, whereas the networking group hosts resources such as Network Policies. Certificate-related resources like Certificate Signing Requests are also grouped under their relevant named groups.



Every API group includes various resources along with associated actions (verbs) such as list, get, create, delete, update, and watch.



For detailed information on the objects in each API group, consult the official Kubernetes API reference documentation.

## Querying the API Server

To retrieve the list of available API groups, access the API server's root endpoint on port 6443:

```
```bash theme={null} curl http://localhost:6443 -k
```

The command returns a JSON response similar to:

```
```json theme={null}
{
 "paths": [
 "/api",
 "/api/v1",
 "/apis",
 "/apis/",
 "/healthz",
 "/logs",
 "/metrics",
 "/openapi/v2",
 "/swagger-2.0.0.json"
]
}
```

When using `curl` without proper authentication, only selected endpoints (like `/version`) may be accessible. Unauthenticated requests to protected endpoints will result in a 403 Forbidden error.

For example, an unauthenticated request may yield:

```
```bash theme={null} curl http://localhost:6443 -k
```

```
```json theme={null}
{
 "kind": "Status",
 "apiVersion": "v1",
 "metadata": {},
 "status": "Failure",
 "message": "forbidden: User \"system:anonymous\" cannot get path \"/\"",
 "reason": "Forbidden",
 "details": {},
 "code": 403
}
```

To fully access the API server, use your certificate files with `curl`:

```
```bash theme={null} curl http://localhost:6443 -k \ --key admin.key \ --cert admin.crt \ --cacert
```

Alternatively, you can use the `kubectl proxy` command, which starts a local HTTP proxy server on port 8001

```
```bash theme={null}
kubectl proxy
```

The output confirms the proxy is running:

```
```bash theme={null} kubectl proxy Starting to serve on 127.0.0.1:8001
```

Now, you can access the API server through the proxy:

```
```bash theme={null}
curl http://localhost:8001 -k
```

The typical response should be:

```
json theme={null} { "paths": ["/api/", "/api/v1", "/apis", "/apis/", "/healthz", "/logs", "/metrics", "/openapi/v2", "/swagger-2.0.0.json"] }
```

Remember that "kube proxy" and "kubectl proxy" serve different purposes. The former facilitates pod-to-pod and service communication within the cluster, while the latter is a local HTTP proxy for accessing the API server.

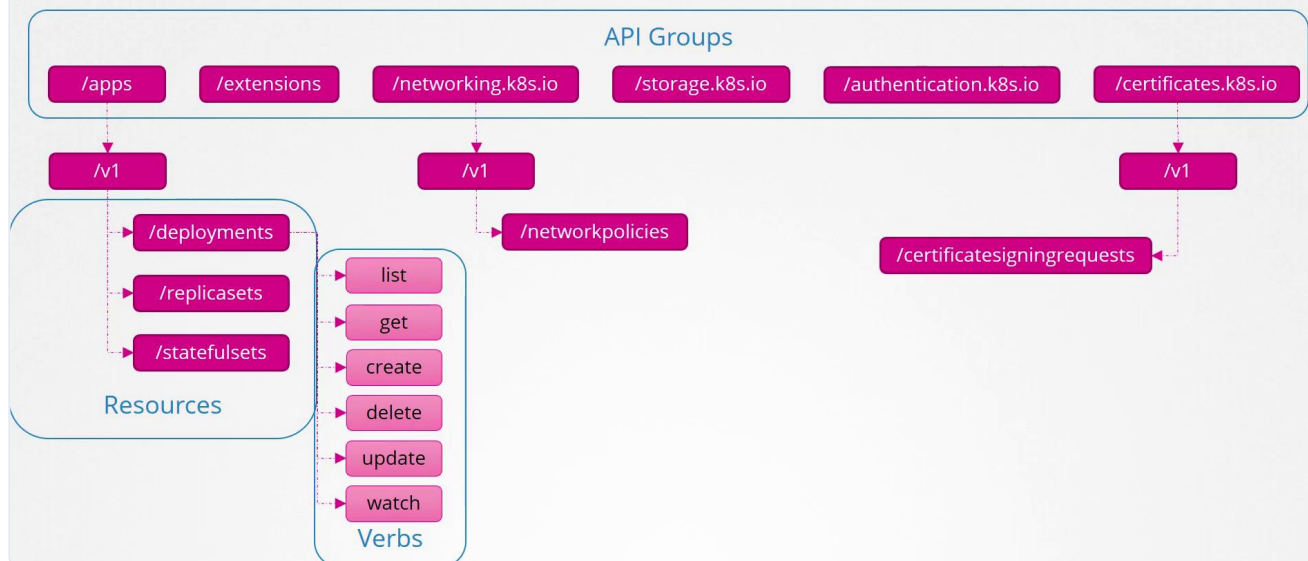
**Kube proxy**  **Kubectl proxy**

## Summary

Kubernetes organizes its resources into various API groups. At the highest level, there is the core API group coupled with multiple named API groups, each containing specific resources and actions. The diagram below outlines the hierarchical structure and relationships between API groups, resources, and associated verbs:

# Key Takeaways

named

**/apis**

In the next section on authorization, we'll explore how these API groups and their associated actions control access to cluster resources.

That's it for this lesson. See you in the next article!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Authorization

This article explores authentication and authorization in Kubernetes, detailing user access, permissions, and various authorization mechanisms like RBAC and external tools.

In this lesson, we explore how authentication allows individuals or machines to gain access to a cluster and how authorization subsequently defines what actions they can perform within that cluster. Once a user gains access, authorization ensures they only have the appropriate permissions for their role. For example, a cluster administrator can view various objects such as Pods, Nodes, and Deployments:

```
```bash theme={null} kubectl get pods NAME READY STATUS RESTARTS AGE nginx 1/1 Running 0 53s
```

```
kubectl get nodes NAME STATUS ROLES AGE VERSION worker-1 Ready 5d21h v1.13.0 worker-2 Ready 5d21h v1.13.0
```

kubec

Administrators have full control, allowing them to create or delete objects like Pods or Nodes. As the cl

Below is an example demonstrating operations executed with limited permissions:

```
```plaintext theme={null}
kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx 1/1 Running 0 53s

kubectl get nodes
NAME STATUS ROLES AGE VERSION
worker-1 Ready <none> 5d21h v1.13.0
worker-2 Ready <none> 5d21h v1.13.0

kubectl delete node worker-2
Node worker-2 Deleted!
```

In contrast, attempting similar operations without sufficient privileges results in the following responses:

```
```bash theme={null} kubectl get pods Error from server (Forbidden): pods is forbidden: User "Bot-1" cannot list "pods"
```

```
kubectl get nodes Error from server (Forbidden): nodes is forbidden: User "Bot-1" cannot get "nodes"
```

kubectl delete node worker-2 Error from server (Forbidden): nodes "worker-2" is forbidden: User "developer" cannot delete resource "nodes"

When sharing a cluster across different organizations or teams using namespaces, authorization restricts

- * Node Authorization
- * Attribute-Based Authorization
- * Role-Based Access Control (RBAC)
- * Webhook Authorization

The Kubernetes API Server is the central component accessed by both management users and internal components

<Frame>

!The image illustrates a Kubernetes node interaction, showing communication between a user, Kube API, and a node.

Requests from kubelets—typically using certificates with names prefixed by "system:node" as part of the system

<Frame>

!The image illustrates the Node Authorizer process in Kubernetes, showing interactions between a user, Kube API, and a node.

<Callout icon="lightbulb" color="#1CB2FE">

Kubernetes supports several authorization strategies to meet diverse security requirements. Always select the appropriate strategy for your environment.

Attribute-Based Authorization

Attribute-based authorization associates specific users or groups with a defined set of permissions. For example,

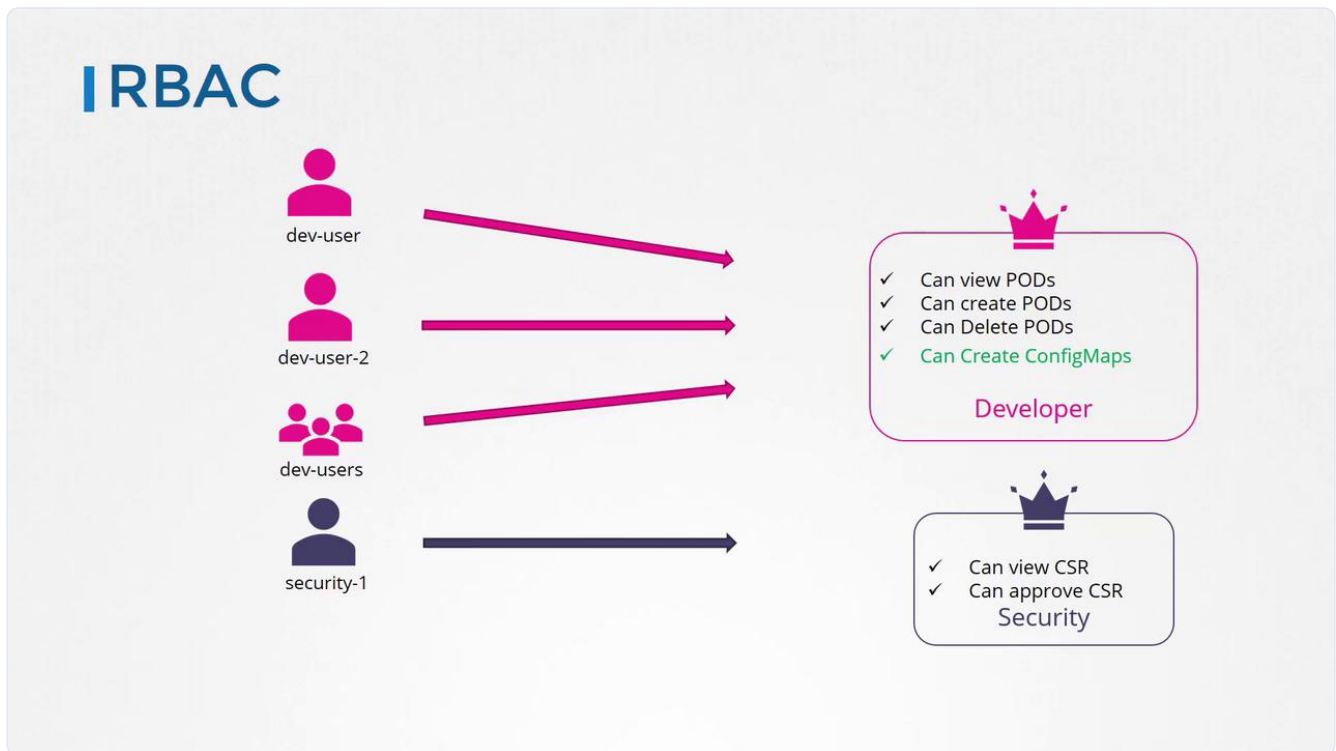
```
```json theme={null}
{"kind": "Policy", "spec": {"user": "dev-user", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"user": "dev-user-2", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"group": "dev-users", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"user": "security-1", "namespace": "*", "resource": "csr", "apiGroup": "*"}}
```

Each time security requirements change, you must manually update this policy file and restart the Kube API Server. This manual process can be tedious and set the stage for more streamlined methods such as Role-Based Access Control (RBAC).

## Role-Based Access Control (RBAC)

RBAC simplifies user permission management by defining roles instead of directly associating permissions with individual users. For example, you can create a "developer" role that encompasses only the necessary permissions for application deployment. Developers are then associated with this role, and modifications in user access can be handled by updating the role, affecting all associated users immediately.

RBAC is considered the standard method for managing access within a Kubernetes cluster. The diagram below provides a visual representation of RBAC across different roles:



Further details on RBAC will be discussed in upcoming lessons.

## External Authorization Mechanisms

If you prefer managing authorization externally rather than with built-in Kubernetes mechanisms, third-party tools like [Open Policy Agent \(OPA\)](#) are an excellent choice. OPA can handle both admission control and authorization by processing user details and access requirements sent via API calls from Kubernetes. Based on OPA's response, access is either granted or denied.

## AlwaysAllow and AlwaysDeny Modes

Kubernetes also supports two basic authorization modes:

- **AlwaysAllow:** Permits all requests without performing any authorization checks.
- **AlwaysDeny:** Denies all requests.

These modes are configured using the `authorization-mode` option on the Kube API Server and are crucial when determining which authorization mechanism is active. In cases where no mode is specified, `AlwaysAllow` is used by default.

Below is an example configuration using `AlwaysAllow`:

```
``bash theme={null} ExecStart=/usr/local/bin/kube-apiserver \ --advertise-address=${INTERNAL_IP} \ --allow-privileged=true \ --apiserver-count=3 \ --authorization-mode=AlwaysAllow \ --bind-address=0.0.0.0 \ --enable-swagger-ui=true \ --etcd-cafile=/var/lib/kubernetes/ca.pem \ --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \ --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \ --etcd-servers=https://127.0.0.1:2379 \ --event-ttl=1h \ --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \ --kubelet-client-certificate=/var/lib/
```

```
kubernetes/apiserver-etcd-client.crt \ --kubelet-client-key=/var/lib/kubernetes/apiserver-etcd-client.key \ --service-
node-port-range=30000-32767 \ --client-ca-file=/var/lib/kubernetes/ca.pem \ --tls-cert-file=/var/lib/kubernetes/
apiserver.crt \ --tls-private-key-file=/var/lib/kubernetes/apiserver.key \ -v=2
```

You can also specify a comma-separated list of multiple authorization modes. For example, to configure node

```
```bash theme={null}
ExecStart=/usr/local/bin/kube-apiserver \
  --advertise-address=${INTERNAL_IP} \
  --allow-privileged=true \
  --apiserver-count=3 \
  --authorization-mode=Node,RBAC,Webhook \
  --bind-address=0.0.0.0 \
  --enable-swagger-ui=true \
  --etcd-cafile=/var/lib/kubernetes/ca.pem \
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \
  --etcd-servers=https://127.0.0.1:2379 \
  --event-ttl=1h \
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.crt \
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \
  --v=2
```

When multiple modes are configured, each request is processed sequentially in the order specified. For example, a user's request is first evaluated by the node authorizer. If the request does not pertain to node-specific actions and is consequently denied, it is then passed to the next module, such as RBAC. Once a module approves the request, further checks are bypassed and the user is granted access.

This lesson provided an in-depth look at authorization in Kubernetes—from basic attribute-based policies to advanced RBAC and external mechanisms. Choosing the right authorization methods is essential for securing your cluster while ensuring users have only the permissions they need.

That concludes this lesson on authorization. Stay tuned for further exploration of role-based access controls and other advanced authorization mechanisms.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Role Based Access Controls

This article explains how to implement Role-Based Access Controls in Kubernetes, including creating roles, role bindings, and verifying permissions.

In this lesson, we dive into Kubernetes Role-Based Access Controls (RBAC) to help you manage permissions effectively. You'll learn how to create roles, bind them to users, and verify permissions within a namespace.

Creating a Role

To define a role, create a YAML file that sets the API version to `rbac.authorization.k8s.io/v1` and the kind to `Role`. In this example, we create a role named **developer** to grant developers specific permissions. The role includes a list of rules where each rule specifies the API groups, resources, and allowed verbs. For resources in the core API group, provide an empty string (`"`) for the `apiGroups` field.

For instance, the following YAML definition grants developers permissions on pods (with various actions) and allows them to create ConfigMaps:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: name: developer rules: - apiGroups: ["" ] resources: ["pods"] verbs: ["list", "get", "create", "update", "delete"] - apiGroups: ["" ] resources: ["ConfigMap"] verbs: ["create"]
```

Create the role by running:

```
```bash theme={null}
kubectl create -f developer-role.yaml
```

Both roles and role bindings are namespace-scoped. This example assumes usage within the default namespace. To manage access in a different namespace, update the YAML metadata accordingly.

Creating a Role Binding

After defining a role, you need to bind it to a user. A role binding links a user to a role within a specific namespace. In this example, we create a role binding named **devuser-developer-binding** that grants the user `dev-user` the **developer** role.

Below is the combined YAML definition for both creating the role and its corresponding binding:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: name: developer rules: -
apiGroups: [""] resources: ["pods"] verbs: ["list", "get", "create", "update", "delete"] - apiGroups: [""] resources:
["ConfigMap"] verbs: ["create"]
```

```
apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: devuser-developer-binding subjects:
- kind: User name: dev-user apiGroup: rbac.authorization.k8s.io roleRef: kind: Role name: developer apiGroup:
rbac.authorization.k8s.io
```

Create the role binding using the command:

```
```bash theme={null}
kubectl create -f devuser-developer-binding.yaml
```

Verifying Roles and Role Bindings

After applying your configurations, it's important to verify that the roles and role bindings have been created correctly.

To list all roles in the current namespace, execute:

```
```bash theme={null} kubectl get roles
```

Example output:

```
```bash theme={null}
NAME          AGE
developer     4s
```

Next, list all role bindings:

```
```bash theme={null} kubectl get rolebindings
```

Example output:

```
```bash theme={null}
NAME                               AGE
devuser-developer-binding 24s
```

For detailed information about the **developer** role, run:

```
```bash theme={null} kubectl describe role developer
```

Sample output:

```
```bash theme={null}
Name:      developer
Labels:    <none>
Annotations: <none>
PolicyRule:
  Resources          Non-Resource URLs  Resource Names  Verbs
  -----
  ConfigMap          []                 []              [create]
  pods               []                 []              [get watch list create delete]
```

To view the specifics of the role binding:

```
```bash theme={null} kubectl describe rolebinding devuser-developer-binding
```

Example output:

```
```bash theme={null}
Name:      devuser-developer-binding
Labels:    <none>
Annotations: <none>
Role:
  Kind:  Role
  Name:  developer
Subjects:
  Kind  Name      Namespace
  ----  -
  User  dev-user
```

Testing Permissions with kubectl auth

You can test whether you have the necessary permissions to perform specific actions by using the `kubectl auth can-i` command. For example, to check if you can create deployments, run:

```
```bash theme={null} kubectl auth can-i create deployments
```

This command might return:

```
```bash theme={null}
yes
```

Similarly, to verify if you can delete nodes:

```
```bash theme={null} kubectl auth can-i delete nodes
```

Expected output:

```
```bash theme={null}
no
```

To test permissions for a specific user without switching user contexts, use the `--as` flag. Although the **developer** role does not permit creating deployments, it does allow creating pods:

```
```bash theme={null} kubectl auth can-i create deployments
```

## Output: yes

---

```
kubectl auth can-i delete nodes
```

## Output: no

---

```
kubectl auth can-i create deployments --as dev-user
```

## Output: no

---

```
kubectl auth can-i create pods --as dev-user
```

## Output: yes

---

You can also specify a namespace in your commands to verify permissions scoped to that particular namespace.

### ## Limiting Access to Specific Resources

In some scenarios, you may want to restrict user access to a select group of resources. For example, if you

Start with a basic role definition without any resource-specific restrictions:

```
```yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
```

Then, update the rule to restrict access solely to the "blue" and "orange" pods:

```
yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: name: developer
rules: - apiGroups: [""] resources: ["pods"] verbs: ["get", "create", "update"] resourceNames:
["blue", "orange"]
```

Conclusion

This lesson provided an in-depth look at implementing Role-Based Access Controls in Kubernetes. You learned how to create roles and role bindings, verify permissions, and restrict access to specific resources. Practicing these exercises will enhance your grasp of RBAC and help you manage Kubernetes security effectively.

For additional details on Kubernetes RBAC, refer to the [Kubernetes Documentation](#) and explore best practices for securing your clusters.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/l1ms.txt> Use this file to discover all available pages before exploring further.

Cluster Roles

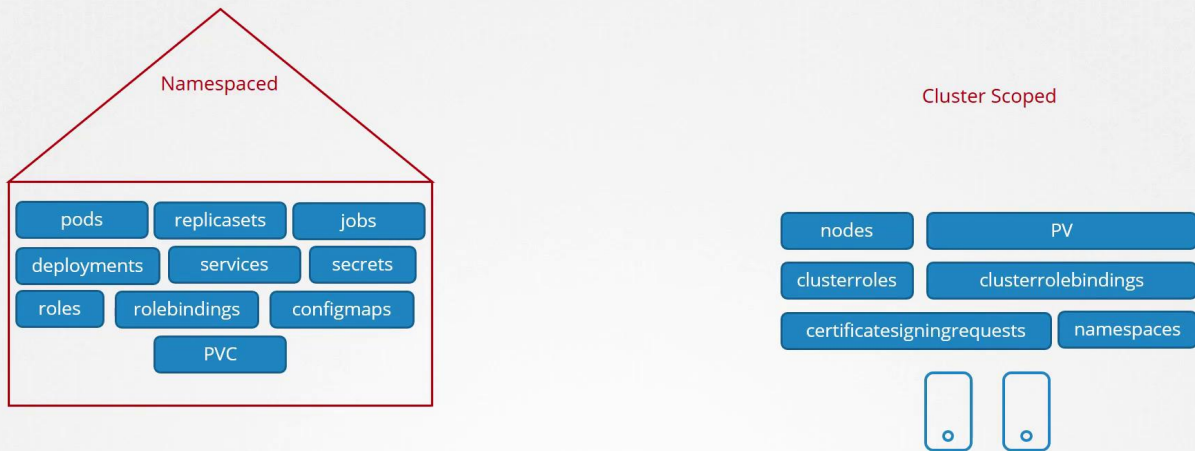
This article introduces cluster roles and bindings for managing permissions across a Kubernetes cluster, distinguishing between namespaced and cluster-scoped resources.

In our previous discussion, we covered roles and role bindings, which are namespace-specific. In this article, we extend that concept by introducing cluster roles and cluster role bindings, allowing you to manage permissions across your entire Kubernetes cluster.

When you create roles and role bindings without specifying a namespace, they are added to the default namespace and only authorize access within that scope. This works well for namespaced resources—such as pods, deployments, and services—but not for cluster-scoped resources. For example, nodes cannot be assigned to a specific namespace (e.g., "node01" cannot belong to the "dev" namespace). Cluster-scoped resources like nodes and persistent volumes are managed at the cluster level, so they require a different approach.

Most resources (such as pods, replica sets, jobs, deployments, services, and secrets) are namespaced. In contrast, cluster-scoped resources, including nodes and persistent volumes, do not belong to any namespace. The following image clearly illustrates the difference between namespaced resources (e.g., pods, services) and cluster-scoped resources (e.g., nodes, cluster roles):

Namespace



To list namespaced and non-namespaced resources, you can use these commands:

```
```bash theme={null} kubectl api-resources --namespaced=true kubectl api-resources --namespaced=false
```

<Callout icon="lightbulb" color="#1CB2FE">

Remember: Roles and role bindings are ideal for namespace-level access, while cluster roles and cluster role bindings are ideal for cluster-level access.

## ## Cluster Roles and Cluster Role Bindings

To authorize cluster-scoped resources, such as nodes and persistent volumes, you need to create cluster roles and cluster role bindings.

For example, you can define a cluster administrator role that grants the ability to list, retrieve, create, and delete cluster resources.

Below is an example of a cluster role definition file named `cluster-admin-role.yaml`. This YAML file defines a ClusterRole object.

```
```yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-administrator
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["list", "get", "create", "delete"]
```

Once the ClusterRole is created, you bind it to a user through a ClusterRoleBinding object. The following example binds the `cluster-administrator` ClusterRole to a user named `cluster-admin`:

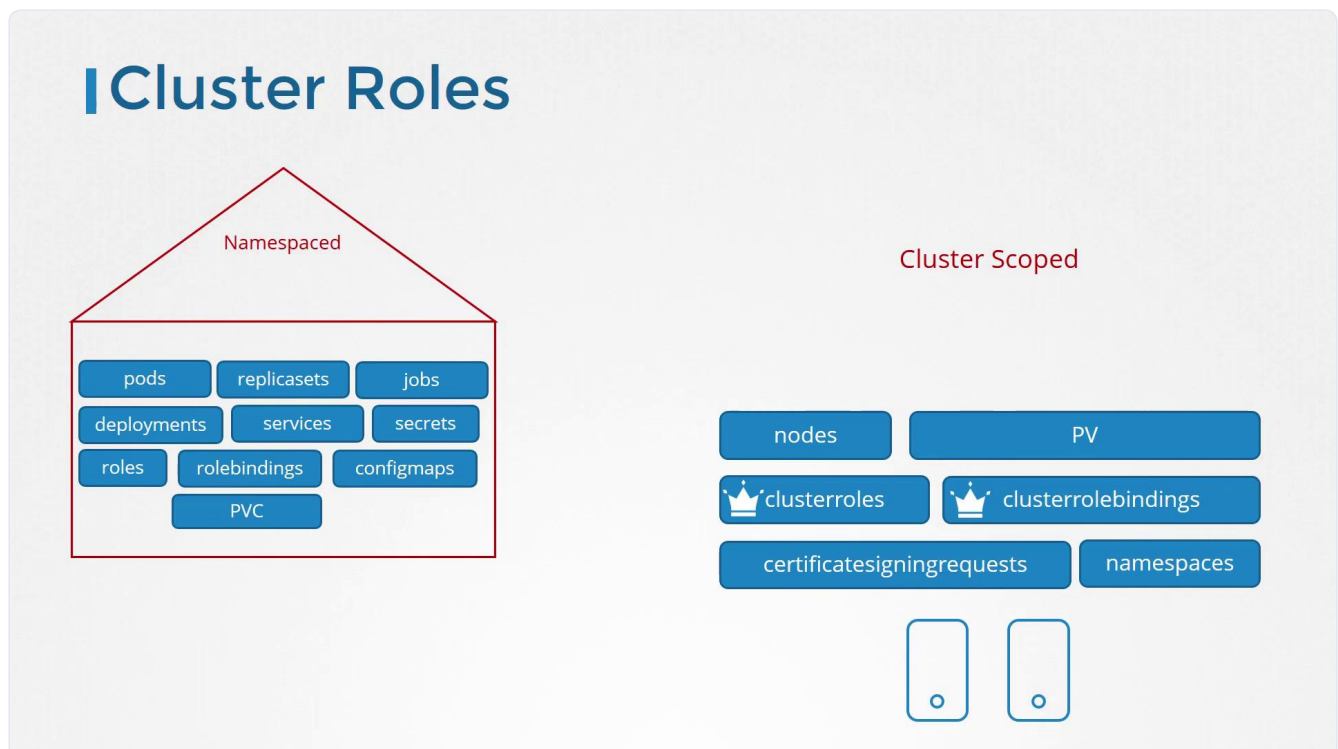
```
yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRoleBinding metadata:
name: cluster-admin-role-binding subjects: - kind: User name: cluster-admin apiGroup:
```

```
rbac.authorization.k8s.io  roleRef:  kind:  ClusterRole  name:  cluster-administrator  apiGroup:  
rbac.authorization.k8s.io
```

Apply these configurations using the `kubectl create` command.

It's important to note that while cluster roles and role bindings are primarily used for cluster-scoped resources, they can also manage access to namespace-scoped resources. When you bind a cluster role that grants permissions on pods, for instance, the user will have access to pods in every namespace—unlike a namespaced role which restricts access to a single namespace.

The following image further distinguishes how cluster roles work for both namespaced and cluster-scoped resources:



Kubernetes provides several default cluster roles when the cluster is initially set up. Be sure to review these defaults to understand the baseline permissions before creating custom roles.

Good luck with managing your cluster roles and enhancing your Kubernetes RBAC!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

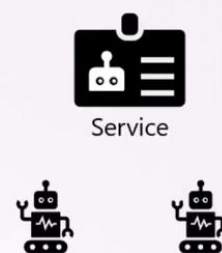
Service Accounts

This guide explains Kubernetes service accounts, their creation, usage, and security enhancements in recent versions.

Welcome to this comprehensive guide on Kubernetes service accounts. In this lesson, we focus on how service accounts function—primarily from an application developer's perspective. While advanced security topics such as authentication, authorization, and role-based access control (RBAC) are covered extensively in the [Kubernetes Administrators Course](#), this guide is dedicated solely to working with service accounts.

There are two main types of accounts in Kubernetes:

- **User Accounts:** Designed for human users like administrators or developers.
- **Service Accounts:** Intended for machine-to-machine interactions or application-specific tasks. For instance, monitoring tools like Prometheus use a service account to query the Kubernetes API for performance metrics, while Jenkins uses one for deploying applications.



Example: A Kubernetes Dashboard Application

Consider an example: "my Kubernetes dashboard," a basic dashboard application built with Python. This application retrieves a list of Pods from a Kubernetes cluster by sending API requests and subsequently displays the results on a web page. To authenticate its API requests, the application uses a dedicated service account.



Creating a Service Account

To create a service account named `dashboard-sa`, run:

```
```bash theme={null} kubectl create serviceaccount dashboard-sa
```

To view all service accounts, use:

```
```bash theme={null}
kubectl get serviceaccount
```

The output will appear similar to:

```
```bash theme={null} NAME SECRETS AGE default 1 218d dashboard-sa 1 4d
```

Upon creation, Kubernetes automatically generates a service account token stored as a Secret and [Links it](#)

```
```bash theme={null}
kubectl describe serviceaccount dashboard-sa
```

Expected output:

```
```bash theme={null} Name: dashboard-sa Namespace: default Labels: Annotations: Image pull secrets:
Mountable secrets: dashboard-sa-token-kbbdm Tokens: dashboard-sa-token-kbbdm Events:
```

To examine the token itself, view the corresponding Secret:

```
```bash theme={null}
kubectl describe secret dashboard-sa-token-kbbdm
```

Sample output:

```
```bash theme={null} Name: dashboard-sa-token-kbbdm Namespace: default Labels: Type: kubernetes.io/
service-account-token Data token: eyJhbGciOiJIUzI1NiIsImtpZCI6Ij... (truncated for privacy)
```

This token serves as the authentication bearer token for accessing the Kubernetes API. For example, using

```
```bash theme={null}
curl https://192.168.56.70:6443/api -k \
--header "Authorization: Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ij...
```

In your custom dashboard application, you would typically place the token into the appropriate configuration field to enable API authentication.

Automatic Mounting of Service Account Tokens

When deploying third-party applications (such as a custom dashboard or Prometheus) on a Kubernetes cluster, you can have Kubernetes automatically mount the service account token as a volume into the Pod. This token is typically available at the path: `/var/run/secrets/kubernetes.io/serviceaccount`.



Every namespace includes a default service account that is automatically injected into Pods. For example, consider the following simple Pod manifest using a custom dashboard image:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: my-kubernetes-dashboard spec: containers: -
name: my-kubernetes-dashboard image: my-kubernetes-dashboard
```

After creating this Pod, running:

```
```bash theme={null}
kubectl describe pod my-kubernetes-dashboard
```

will reveal a volume mounted from a Secret (usually named something like `default-token-xxxx`). You might see an excerpt similar to:

```
```bash theme={null} Name: my-kubernetes-dashboard Namespace: default Status: Running IP: 10.244.0.15
Containers: nginx: Image: my-kubernetes-dashboard Mounts: /var/run/secrets/kubernetes.io/serviceaccount from
default-token-j4hkv (ro) Volumes: default-token-j4hkv: Type: Secret (a volume populated by a Secret)
SecretName: default-token-j4hkv Optional: false
```

Inside the Pod, listing the contents of the service account directory shows files such as the `token` file:

```
```bash theme={null}
kubectl exec -it my-kubernetes-dashboard -- ls /var/run/secrets/kubernetes.io/serviceaccount
kubectl exec -it my-kubernetes-dashboard -- cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

Using a Different Service Account

By default, Pods use the `default` service account. To assign a different service account—like the previously created `dashboard-sa`—update your Pod definition to include the `serviceAccountName` field:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: my-kubernetes-dashboard spec:
serviceAccountName: dashboard-sa containers: - name: my-kubernetes-dashboard image: my-kubernetes-
dashboard
```

<Callout icon="lightbulb" color="#1CB2FE">

Remember that you cannot modify the service account of an existing Pod. To use a new service account, delete the existing Pod and create a new one.

</Callout>

After deploying the updated manifest, running:

```
```bash theme={null}
kubectl describe pod my-kubernetes-dashboard
```

will show that the new service account is now in effect, with volume mounting information reflecting the token for `dashboard-sa` (e.g., `dashboard-sa-token-kbbdm`).

If you wish to disable the automatic mounting of the service account token, set `automountServiceAccountToken` to `false` in the Pod specification:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: my-kubernetes-dashboard spec:
automountServiceAccountToken: false containers: - name: my-kubernetes-dashboard image: my-kubernetes-
dashboard
```

## ## Changes in Kubernetes Versions 1.22 and 1.24

Prior to Kubernetes v1.22, service account tokens were automatically mounted from Secrets without an explicit request.

Below is an example Pod definition using a projected volume sourced from the TokenRequest API:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: default
spec:
  containers:
    - image: nginx
      name: nginx
      volumeMounts:
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
          name: kube-api-access-6mtg8
          readOnly: true
  volumes:
    - name: kube-api-access-6mtg8
      projected:
        defaultMode: 420
        sources:
          - serviceAccountToken:
              expirationSeconds: 3607
              path: token
          - configMap:
              name: kube-root-ca.crt
              items:
                - key: ca.crt
                  path: ca.crt
          - downwardAPI:
              items:
                - fieldRef:
                    apiVersion: v1
                    fieldPath: metadata.namespace
```

Starting with Kubernetes v1.24, Kubernetes no longer automatically creates non-expiring service account tokens stored as Secrets. Instead, after creating a new service account, you must generate a token explicitly with:

```
```bash theme={null} kubectl create token dashboard-sa
```

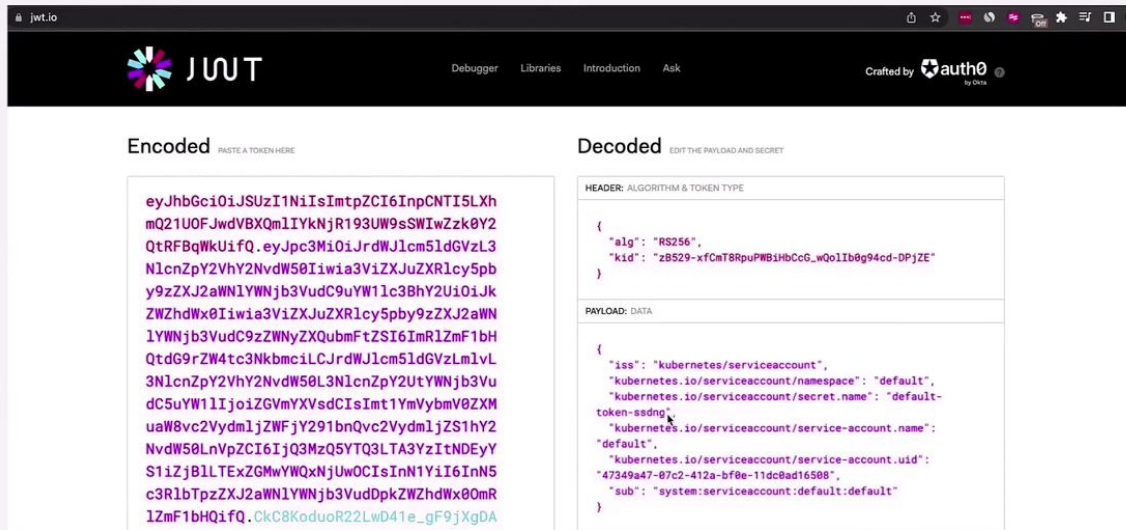
This command produces a token with an expiry (by default, one hour from creation). You can verify and decode the token with:

```
```bash theme={null}
jq -R 'split(".") | select(length > 0) | .[0] | @base64 | fromjson' <<< <TOKEN>
```

If necessary (though not recommended), you can still create a non-expiring token by manually creating a Secret. Ensure the service account exists first:

```
yaml theme={null} apiVersion: v1 kind: Secret metadata: name: mysecretname annotations:
kubernetes.io/service-account.name: dashboard-sa type: kubernetes.io/service-account-token
```

It is highly recommended to use the TokenRequest API to generate tokens, as API-generated tokens provide additional security features such as expiry, audience restrictions, and improved manageability.



v1.22

KEP 1205 - Bound Service Account Tokens

Background

Kubernetes already provisions JWTs to workloads. This functionality is on by default and thus widely deployed. The current workload JWT system has serious issues:

1. Security: JWTs are not audience bound. Any recipient of a JWT can masquerade as the presenter to anyone else.
2. Security: The current model of storing the service account token in a Secret and delivering it to nodes results in a broad attack surface for the Kubernetes control plane when powerful components are run - giving a service account a permission means that any component that can see that service account's secrets is at least as powerful as the component.
3. Security: JWTs are not time bound. A JWT compromised via 1 or 2, is valid for as long as the service account exists. This may be mitigated with service account signing key rotation but is not supported by client-go and not automated by the control plane and thus is not widely deployed.
4. Scalability: JWTs require a Kubernetes secret per service account.

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/1205-bound-service-account-tokens#background>

v1.22

KEP 1205 - Bound Service Account Tokens



TokenRequestAPI

- ✓ Audience Bound
- ✓ Time Bound

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/1205-bound-service-account-tokens#background>

Service account token Secrets

A `kubernetes.io/service-account-token` type of Secret is used to store a token credential that identifies a service account.

Since 1.22, this type of Secret is no longer used to mount credentials into Pods, and obtaining tokens via the `TokenRequest` API is recommended instead of using service account token Secret objects. Tokens obtained from the `TokenRequest` API are more secure than ones stored in Secret objects, because they have a bounded lifetime and are not readable by other API clients. You can use the `kubectl create token` command to obtain a token from the `TokenRequest` API.

You should only create a service account token Secret object if you can't use the `TokenRequest` API to obtain a token, and the security exposure of persisting a non-expiring token credential in a readable API object is acceptable to you.

<https://kubernetes.io/docs/concepts/configuration/secret/#service-account-token-secrets>

Summary

- **Service Accounts vs. User Accounts:** Service accounts are meant for applications (or machines), whereas user accounts are for human users.
- **Token Generation:** Creating a service account automatically generates a token stored in a Secret, which is used for API authentication.

- **Automatic Token Mounting:** Pods can automatically mount the service account token at `/var/run/secrets/kubernetes.io/serviceaccount`, though this behavior can be modified.
- **Enhanced Security:** Since Kubernetes v1.22, tokens are generated using the TokenRequest API, making them audience-bound, time-bound, and more secure.
- **Kubernetes v1.24 Changes:** With v1.24, Kubernetes no longer provisions non-expiring tokens automatically via Secrets; use the `kubectl create token` command to generate tokens as needed.

For additional insights, refer to the Kubernetes enhancement proposals and the official documentation on service accounts and tokens.

References

KEP 1205 - Bound Service Account Tokens

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/1205-bound-service-account-tokens>

KEP-2799: Reduction of Secret-based Service Account Tokens

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/2799-reduction-of-secret-based-service-account-token>

<https://kubernetes.io/docs/concepts/configuration/secret/#service-account-token-secrets>
<https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/>

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Image Security

This article covers best practices for securing container images during deployment, including naming conventions, private registries, and Kubernetes configuration.

In this lesson, we explore best practices for securing container images throughout the deployment process. You will learn about image naming conventions, securing image repositories, and configuring your pods to pull images from trusted sources. We will illustrate these concepts using several pod examples that deploy various applications such as web apps, databases, and Redis caches.

Understanding Container Image Naming

Let's start by examining a simple pod definition file that deploys an Nginx container:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx-pod spec: containers: - name: nginx image: nginx
```

Notice the image name "nginx". This follows Docker's image naming convention. When a repository name is provided, the image is pulled from that repository.

If you create your own account and build custom images, you should update the image name accordingly. For example, if you have a repository named "my-account/nginx", the image name should be "my-account/nginx".

```
```yaml theme={null} image: your-account/nginx
```

By default, Docker pulls images from Docker Hub (with the DNS name docker.io) if no other registry is specified. The registry is a centralized storage where images are pushed during creation or updates, and subsequently pulled during deployment.

Private Registry Usage

For projects that require enhanced security and privacy, you might opt for private registries. Many popular cloud service providers—such as AWS, Azure, and GCP—offer private registries built into their platforms. Alternatively, tools like [Google Container Registry](https://gcr.io) (gcr.io) are frequently used for Kubernetes-related images and testing purposes.

When referencing an image from a private registry, the full image path should be specified. For example:

```
```yaml theme={null} image: docker.io/library/nginx
```

### ### Authentication for Private Registries

Accessing private repositories requires prior authentication. Start by logging into your private registry

```
```bash theme={null}
docker login private-registry.io
```

After you provide your credentials, you should see a confirmation similar to this:

```
```plaintext theme={null} Login with your Docker ID to push and pull images from Docker Hub. If you don't have a
Docker ID, head over to https://hub.docker.com to create one. Username: registry-user Password: WARNING!
Your password will be stored unencrypted in /home/vagrant/.docker/config.json. Login Succeeded
```

### ## Configuring Kubernetes Pods for Private Registries

To pull an image from a private registry within a pod, specify the full image path in your pod definition

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
```

Since Kubernetes worker nodes rely on the Docker runtime for image retrieval, they must be provided with the appropriate credentials. This is achieved by creating a Kubernetes secret of type Docker registry. Execute the following command to create the secret:

```
```bash theme={null} kubectl create secret docker-registry regcred \ --docker-server=private-registry.io \ --docker-
username=registry-user \ --docker-password=registry-password \ --docker-email=registry-user@org.com
```

Once the secret is created, reference it in your pod specification using the `imagePullSecrets` section:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
  imagePullSecrets:
    - name: regcred
```

When the pod is created, the Kubelet on the worker node will use the credentials stored in the secret to authenticate and pull the image from your private registry.

Summary

This lesson covered key aspects of container image security by demonstrating:

- The importance of proper image naming conventions.
- How to designate public and private repositories.
- Steps for authenticating with private registries.
- Configuring Kubernetes pods with image pull secrets.

By following these practices, you ensure that your applications are deployed using secure and trusted container images. Now, put your understanding into practice and work with secure images in your own projects.

For more information, check out the [Kubernetes Documentation](#) and best practices guides on container security.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Security Contexts

This lesson explains how to secure containers in Kubernetes by configuring security settings at the pod and container levels.

Hello, and welcome to this lesson on Security Contexts in Kubernetes. My name is Mumshad Mannambeth, and in this guide, we'll explain how you can secure your containers by configuring security settings at both the pod and container levels.

When you run a Docker container, you have the flexibility to define various security parameters. For example, you can set a specific user ID or modify Linux capabilities. Consider the following Docker commands:

```
```bash theme={null} docker run --user=1001 ubuntu sleep 3600 docker run --cap-add MAC_ADMIN ubuntu
```

These configurations help manage the security of Docker containers, and similar settings are available in

In Kubernetes, containers are always encapsulated in pods. You can define security settings either at the

Below is an example of a pod definition that configures the security context at the container level, using

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        runAsUser: 1000
        capabilities:
          add: ["MAC_ADMIN"]
```

This configuration instructs Kubernetes to run the container as user ID 1000 and adds the `MAC_ADMIN` capability. If your goal is to enforce these security settings across all containers within a single pod, you can define the security context at the pod level instead.

For more hands-on practice with viewing, configuring, and troubleshooting security contexts in Kubernetes, check out the available lab resources.

By leveraging security contexts, you enhance the security posture of your containerized applications in Kubernetes. For additional guidance, you may find the following resources useful:

- [Kubernetes Documentation](#)
- [Kubernetes Concepts: Security Contexts](#)

Thank you for following along, and happy securing!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Network Policies

This lesson reviews networking and security concepts, focusing on how network policies function within a Kubernetes environment.

Welcome to this lesson on network policies. This guide will review essential networking and security concepts before diving into how network policies function within a Kubernetes environment.

Traffic Flow Example

Consider a simple application configuration consisting of a web server, an API server, and a database server. The traffic flow is as follows:

1. A user sends a request to the web server on port 80.
2. The web server forwards this request to the API server on port 5000.
3. The API server retrieves data from the database server on port 3306, then sends the response back to the user.

There are two main types of network traffic involved:

- **Ingress traffic:** Incoming traffic. For example, user requests arriving at the web server on port 80.
- **Egress traffic:** Outgoing traffic. For example, requests sent from the web server to the API server.

In our diagrams, a solid arrow indicates the direction of the originating traffic (either ingress or egress), while a dotted arrow represents the response flow, which is typically not controlled by network policies.

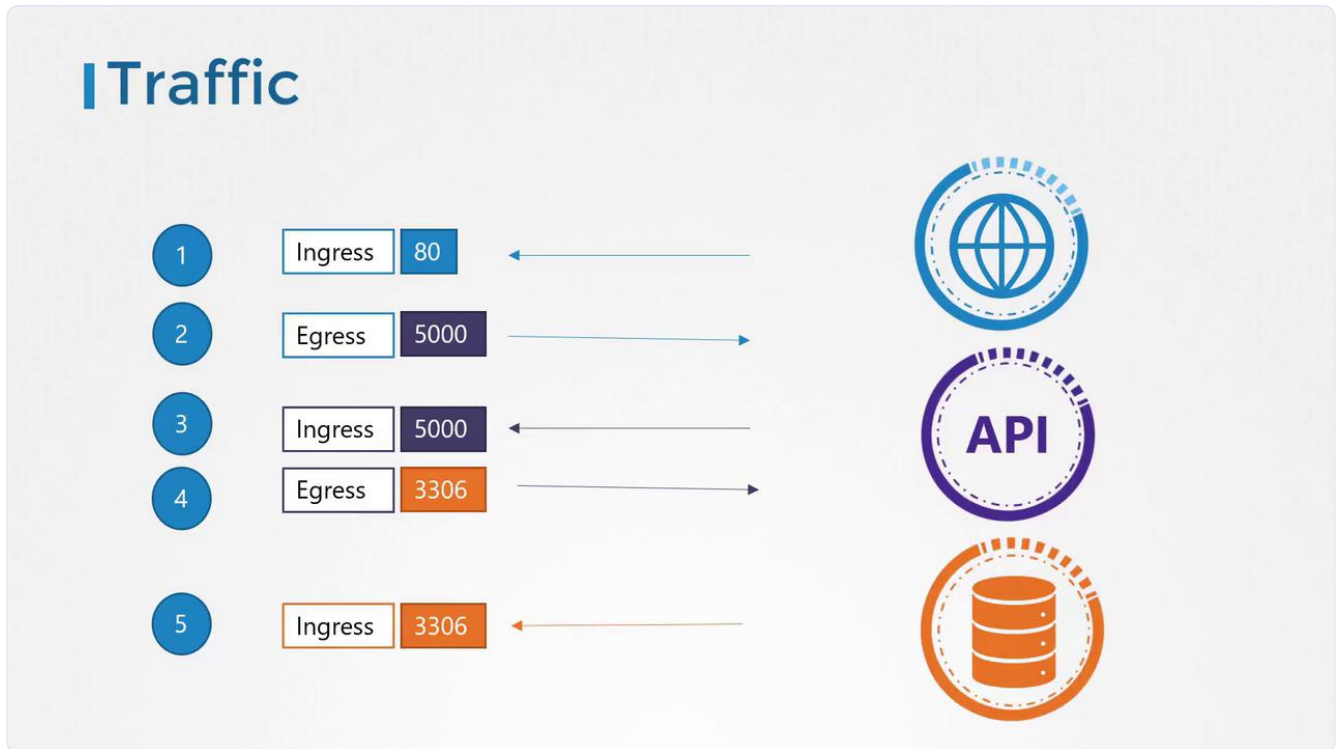
For clarity:

- The **API server** receives ingress traffic from the web server (port 5000) and sends out egress traffic to the database server (port 3306).
- The **database server** receives ingress traffic on port 3306 from the API server.

To support this traffic flow, the following rules must be established:

- An ingress rule for the web server to allow HTTP traffic on port 80.
- An egress rule for the web server to permit traffic to port 5000 on the API server.
- An ingress rule for the API server to accept traffic on port 5000.
- An egress rule for the API server to allow traffic to port 3306 on the database server.

- An ingress rule for the database server to allow traffic on port 3306.

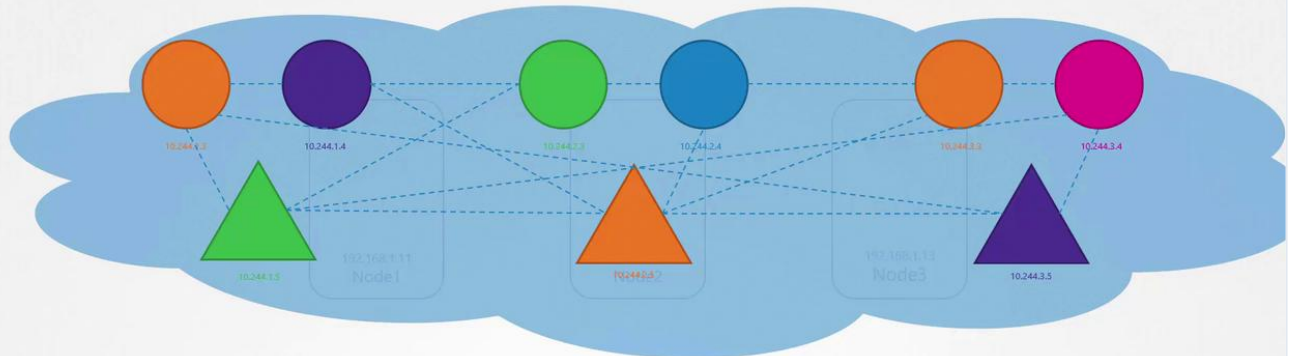


Network Security in Kubernetes

In a Kubernetes cluster, nodes host pods and services, each assigned a unique IP address. A crucial capability of Kubernetes is that pods can communicate with one another without extra configuration—such as setting up custom routes. Typically, all pods reside in a virtual private network (VPN) that spans the entire cluster, allowing them to interact using pod IPs, pod names, or configured services.

By default, Kubernetes employs an "all-allow" rule permitting any pod to communicate with every other pod or service within the cluster.

Network Security



"All Allow"

Now, consider the earlier scenario with three pods: one for the front-end web server, one for the API server, and one for the database server. Services facilitate communication between these pods and external users, while the default configuration allows free communication across the cluster.

Restricting Communication with Network Policies

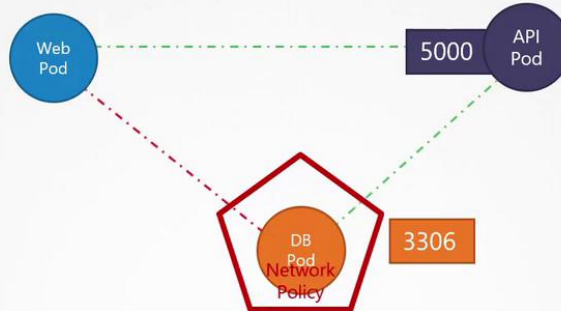
If your security requirements dictate that the front-end web server should not communicate directly with the database server, you can enforce this by implementing a network policy. For example, you might create a policy that only permits the API server to interact with the database server.

A network policy in Kubernetes is defined as an object, which you attach to one or more pods using labels and selectors. In this scenario, the policy would only allow ingress traffic from the API pod on port 3306 while blocking all other sources from accessing the database pod.

Network Policy



80



Implementing a Network Policy

To apply a network policy, you assign labels to pods and define matching selectors in the network policy object. For example, consider this snippet used to select the database pod:

```
```yaml theme={null} podSelector: matchLabels: role: db
```

This configuration ensures the network policy only applies to pods labeled with `role: db`. Next, you def

```
<Frame>
 ![The image illustrates a network policy allowing ingress traffic from an API pod to a DB pod on port 3306]
</Frame>
```

Below is the complete network policy configuration:

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              name: api-pod
      ports:
        - protocol: TCP
          port: 3306
```

In this configuration:

- The `podSelector` targets the database pod through its label.
- The `policyTypes` field specifies that only ingress traffic is affected.
- The ingress rule allows traffic specifically from pods with the label `name: api-pod` on TCP port 3306.

Keep in mind that isolation only applies to the traffic explicitly defined under `policyTypes`. Unspecified traffic is automatically allowed by default.

Enabling Network Policies in Kubernetes

To enforce the network policy, execute the following command:

```
bash theme={null} kubectl create -f db-policy.yaml
```

Network policies are enforced by the cluster's networking solution. While solutions like Kube-router, Calico, Romana, and Weave Net support network policies, Flannel does not enforce them. Creating policies with Flannel will not produce an error, but they won't be applied. Always consult your network solution's documentation to verify its support for network policies.

Conclusion

This lesson provided an overview of network policies by demonstrating a basic application setup and illustrating how to restrict pod communication within a Kubernetes cluster. For further practice, be sure to review the Kubernetes documentation and engage in coding challenges to deepen your understanding of network policies.

Additional Resources

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Developing network policies

This article explores advanced network policy configurations in Kubernetes to secure database pods by controlling ingress and egress traffic.

In this article, we explore advanced network policy configurations within Kubernetes. We will use our familiar web API and database pods to demonstrate how to secure your database pod by allowing only authorized access. Specifically, only the API pod will access the database pod on port 3306, while other pods (such as the web pod) remain unrestricted. By default, Kubernetes permits all traffic between pods, so explicit network policies are necessary to enforce these security measures.

Blocking All Ingress Traffic to the Database Pod

To begin, we block all incoming traffic to the database pod. This is achieved by creating a network policy that targets the database pod using labels and selectors. In our example, the database pod is labeled `role: db`. The following YAML file defines a policy that denies all ingress traffic by default since no specific ingress rules are provided:

```
```yaml theme={null} apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: db-policy spec: podSelector: matchLabels: role: db policyTypes: - Ingress
```

<Callout icon="lightbulb" color="#1CB2FE">

If no ingress rules are specified, Kubernetes treats the policy as a complete block for incoming traffic.

</Callout>

### ## Allowing Ingress Traffic from the API Pod

The next step is to allow ingress traffic from the API pod on port 3306. Because responses to permitted

For instance, to allow traffic only from pods labeled `name: api-pod` within namespaces labeled `prod` and

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              name: api-pod
          namespaceSelector:
            matchLabels:
              name: prod
        - ipBlock:
            cidr: 192.168.5.10/32
  ports:
    - protocol: TCP
      port: 3306
```

In this configuration, the first entry under the `from` section restricts traffic to API pods in the production namespace (an AND condition). The second entry allows an external backup server by specifying its IP block.

Combining pod selectors with namespace selectors ensures that the rule applies only to the intended pods within the correct namespace.

Configuring Egress Traffic

In scenarios where the database pod must initiate outbound connections (for example, sending backups to an external server), an egress rule becomes necessary. To support both ingress and egress traffic, include `Egress` in the `policyTypes` and specify an egress rule.

The revised policy below permits the database pod to send traffic to a backup server at IP `192.168.5.10` on port `80` while still restricting all other outbound connections:

```
yaml theme={null} apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: db-policy
spec: podSelector: matchLabels: role: db policyTypes: - Ingress - Egress ingress: - from: -
```

```
podSelector: matchLabels: name: api-pod namespaceSelector: matchLabels: name: prod ports: -
protocol: TCP port: 3306 egress: - to: - ipBlock: cidr: 192.168.5.10/32 ports: - protocol: TCP
port: 80
```

Ensure that your egress rules cover all required outbound connections. Missing an egress rule may inadvertently block critical communication between your services.

Summary of Network Policy Configuration

Configuration Aspect	Description	YAML Reference
Blocking Traffic	Deny all ingress traffic to the database pod by default.	Initial policy with <code>podSelector</code> for <code>role: db</code> .
Allowing Ingress	Permit API pod access on port 3306 by combining pod and namespace selectors. Also allow a specific external IP.	Ingress rule with pod and namespace selectors plus <code>ipBlock</code> .
Configuring Egress	Enable the database pod to send outbound traffic to an external backup server on port 80.	Egress rule addition with <code>ipBlock</code> for backup server.

Experiment in your lab environment with these network policy configurations to gain hands-on experience with Kubernetes security practices. Understanding how selectors and rule ordering interact in your network policies will help you optimize traffic flow and maintain a secure architecture.

For additional resources on Kubernetes networking and security, visit the following links:

- [Kubernetes Documentation](#)
- [Kubernetes Networking Concepts](#)
- [Kubernetes API Reference](#)

Happy networking, and see you in the next article!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

2025 Updates Custom Resource Definition CRD

This guide explores Custom Resource Definitions in Kubernetes, detailing standard resources, custom resources like FlightTicket, and the need for custom controllers.

In this guide, we dive into Custom Resource Definitions (CRDs) in Kubernetes, beginning with an overview of standard Kubernetes resources and controllers before extending these principles to custom resources like our FlightTicket example.

Understanding Standard Kubernetes Resources and Controllers

Kubernetes relies on built-in controllers to manage standard resources. For instance, when you create a Deployment, Kubernetes stores the desired state in its etcd data store and automatically manages related ReplicaSets and Pods. The deployment controller continuously monitors the Deployment and ensures that the cluster state matches the desired configuration. Creating a Deployment with three replicas will result in three Pods being deployed.

Below is an example YAML file defining a Deployment:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: myapp-deployment spec: replicas: 3 selector: matchLabels: type: front-end template: metadata: name: myapp-pod labels: type: front-end spec: containers: - image: nginx
```

After saving the above content as `deployment.yml`, run the following commands to create, view, and delete

```
```bash theme={null}
kubectl create -f deployment.yml
# Output:
kubectl get deployments
# Output:
# NAME                DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
kubectl delete -f deployment.yml
# Output:
# deployment "myapp-deployment" deleted
```

The deployment controller (implemented in Go within the Kubernetes source code) handles the creation of a ReplicaSet when a new Deployment is detected. The ReplicaSet then creates the required Pods based on the

Deployment's specification. This dynamic process ensures that the actual state of the cluster continuously converges with the desired state.

Custom Resources and Controllers: The Flight Ticket Example

Building on the standard resource management, you can extend Kubernetes by defining custom resources. Imagine a scenario where you want to manage flight ticket bookings directly in Kubernetes. With a custom resource called `FlightTicket`, you can create objects representing flight ticket bookings, list them, and delete them as needed.

FlightTicket Object Definition

Below is an example YAML file that defines a `FlightTicket` object:

```
```yaml theme={null} apiVersion: flights.com/v1 kind: FlightTicket metadata: name: my-flight-ticket spec: from: Mumbai to: London number: 2
```

To create and manage this custom resource, execute the following commands:

```
```bash theme={null}
kubectl create -f flightticket.yml
# Output:
kubectl get flightticket
# Output:
# NAME                STATUS
kubectl delete -f flightticket.yml
# Output:
# flightticket "my-flight-ticket" deleted
```

At this stage, the `FlightTicket` object is stored in etcd; however, it does not trigger any actions. To automate operations—such as interfacing with an external API (e.g., `bookflight.com/API`) to book or cancel a ticket—you need a custom controller.

Custom Controller for FlightTicket

A custom controller, typically written in Go, monitors `FlightTicket` objects. When a `FlightTicket` is created, updated, or deleted, the controller calls an external API to perform actions such as booking or canceling a flight. Below is a streamlined Go snippet to illustrate the controller's logic:

```
```go theme={null} package flightticket

import (// Imports omitted for brevity)

var controllerKind = apps.SchemeGroupVersion.WithKind("FlightTicket")

// Run begins watching and syncing FlightTicket resources. func (dc *FlightTicketController) Run(workers int, stopCh <-chan struct{}) { // Controller loop implementation here }

// callBookFlightAPI books a flight ticket when a FlightTicket resource is created. func (dc *FlightTicketController) callBookFlightAPI(obj interface{}) { // API calling logic here }
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
 Without this custom controller, FlightTicket objects remain static data in etcd, and no automated flight
</Callout>
```

### ### Handling Resource Creation Errors

If you create a FlightTicket object before Kubernetes is aware of its type, you will encounter an error:

```
```bash theme={null}
kubectl create -f flightticket.yml
# Output:
# no matches for kind "FlightTicket" in version "flights.com/v1"
```

This error appears because Kubernetes does not recognize the FlightTicket resource type. To resolve this, you must first establish a Custom Resource Definition (CRD) for FlightTicket.

Defining a Custom Resource with a CRD

A Custom Resource Definition (CRD) informs Kubernetes about a new resource type, detailing its metadata, scope (namespaced or cluster-scoped), API group, naming conventions (singular, plural, and short names), supported versions, and a validation schema using OpenAPI v3.

Below is a sample CRD for the FlightTicket resource:

```
```yaml theme={null} apiVersion: apiextensions.k8s.io/v1 kind: CustomResourceDefinition metadata: name:
flighttickets.flights.com spec: group: flights.com scope: Namespaced names: plural: flighttickets singular:
flightticket kind: FlightTicket shortNames: - ft versions: - name: v1 served: true storage: true schema:
openAPIV3Schema: type: object properties: spec: type: object properties: from: type: string to: type: string
number: type: integer minimum: 1
```

Save this content as `flightticket-custom-definition.yml` and apply it to your Kubernetes cluster using:

```
```bash theme={null}
kubectl create -f flightticket-custom-definition.yml
# Output:
# customresourcedefinition "flighttickets.flights.com" created
```

Once the CRD is successfully created, Kubernetes can recognize and store FlightTicket objects. However, remember that without the corresponding custom controller, FlightTicket objects will remain as passive data entries.

Summary

In this guide, we:

- Explored how standard Kubernetes resources, such as Deployments, are managed using built-in controllers.
- Demonstrated the creation, listing, and deletion of a Deployment using a YAML file and kubectl commands.

Introduced custom resources with a FlightTicket example, emphasizing the need for a Custom Resource Definition (CRD) and a custom controller to automate actions.

Future articles will cover the implementation of a custom controller to automatically process FlightTicket events and integrate with external APIs.

For more details on Kubernetes resources and controllers, check out the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

2025 Updates Custom Controllers

This lesson covers developing custom controllers for Kubernetes, focusing on monitoring FlightTicket objects and integrating with a flight booking API.

In this lesson, we delve into developing custom controllers for Kubernetes. Building on our previous work, we have already defined a custom resource and created FlightTicket objects with data stored in etcd. The next step is to continuously monitor these objects and perform corresponding actions—such as calling a flight booking API to book, modify, or cancel flight tickets. This process is the core function of a custom controller.

A controller is a process running in a loop that monitors the Kubernetes cluster and reacts to changes in specific objects (in this case, FlightTicket resources). Consider the following FlightTicket definition as a starting point:

```
```yaml theme={null} apiVersion: flights.com/v1 kind: FlightTicket metadata: name: my-flight-ticket spec: from: Mumbai to: London number: 2
```

You can create the resource using:

```
```bash theme={null}
kubectl create -f flightticket.yml
```

The command output confirms the creation:

```
```bash theme={null} flightticket "my-flight-ticket" created
```

To check the status of your FlightTicket, run:

```
```bash theme={null}
kubectl get flightticket
```

Expected output:

```
```text theme={null} NAME STATUS My-flight-ticket Pending
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
While it is possible to write a controller in a language like Python, challenges such as managing expensive dependencies and stateful logic can be complex.
</Callout>
```

Developing the controller in Go using the Kubernetes Go client offers a more efficient approach. It provides a rich ecosystem of libraries and tools.

### ## Getting Started with the Custom Controller

To begin, clone the SampleController repository from GitHub. Ensure that the Go programming language is installed on your machine.

```
```bash theme={null}
git clone https://github.com/kubernetes/sample-controller.git
```

The terminal should display output similar to:

```
```text theme={null} Cloning into 'sample-controller'... Resolving deltas: 100% (15787/15787), done.
```

Change into the cloned directory:

```
```bash theme={null}
cd sample-controller
```

Customize the file `controller.go` with your specific business logic. One important function within your controller might involve making a call to the flight booking API after detecting changes in `FlightTicket` objects.

Once you've incorporated your custom logic, build the controller with the command:

```
```bash theme={null} go build -o sample-controller .
```

During the build process, you may encounter messages such as:

```
```text theme={null}
go: downloading k8s.io/client-go v0.0.0-20211001003700-dbfa30b9d908
go: downloading golang.org/x/text v0.3.6
```

Run the controller by specifying the path to your kubeconfig file:

```
```bash theme={null} ./sample-controller -kubeconfig=$HOME/.kube/config
```

You should see log messages that confirm the event handlers are being set up and that the `FlightTicket` controller is starting.

```
```text theme={null}
I1013 02:11:07.489479 40117 controller.go:115] Setting up event handlers
I1013 02:11:07.489701 40117 controller.go:156] Starting FlightTicket controller
```

Inside the Controller Code

Your controller code might include sections like the following:

```
```go theme={null} package flightticket
```

```
var controllerKind = apps.SchemeGroupVersion.WithKind("Flightticket")
```

```
// Code hidden for brevity

// Run begins watching and syncing. func (dc *FlightTicketController) Run(workers int, stopCh <-chan struct{}) {}

// Code hidden for brevity

// callBookFlightAPI triggers the flight booking process. func (dc *FlightTicketController) callBookFlightAPI(obj
interface{}) {} ``
```

The controller leverages the specified kubeconfig file to authenticate with the Kubernetes API server. It watches for changes to FlightTicket objects and, upon detecting a creation or modification, it executes your custom logic—such as interfacing with the flight booking API—to reconcile the desired state.

Ensure your API calls are handled efficiently to avoid timeouts and performance issues in production environments.

## Deploying Your Controller

---

Once your custom controller is fully verified and functional, you may want to package it as a Docker image and deploy it within your Kubernetes cluster as a pod or Deployment. This approach streamlines updates by avoiding the need for manual rebuilding and restarting of the controller with every change.

## Summary

---

This lesson provided a comprehensive overview of building a custom controller in Kubernetes. It discussed:

- How to define custom resources for FlightTickets.
- Monitoring and reacting to changes using controllers.
- Building the controller in Go with the built-in queuing and caching support provided by the Kubernetes Go client.
- Deploying your controller in a production environment.

Understanding how Custom Resource Definitions (CRDs) and controllers work together is fundamental, especially since exam questions may cover creating CRDs, managing CRD files, or interacting with existing controllers. Operators, which extend controller functionality further, are another exciting area to explore.

For more details on Kubernetes concepts, check out these resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [GitHub Repository for SampleController](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# 2025 Updates Operator Framework

This article explores the operator framework, highlighting its role in simplifying Kubernetes resource deployment and management.

In this article, we dive into the operator framework and explore how it simplifies the deployment and management of Kubernetes resources. Previously, we discussed creating a Custom Resource Definition (CRD) and a custom controller to handle resource-specific logic. Traditionally, these components are deployed separately: you first create the CRD and its related resources, and then deploy the controller as a pod or as part of a deployment. With the operator framework, you can package both components into a single deployable entity.

When you deploy the flight operator, it automatically creates the Custom Resource Definition, provisions the required resources, and deploys the custom controller as a Deployment. Consider the following example:

```
```yaml theme={null}
```

flightticket-custom-definition.yml

```
apiVersion: apiextensions.k8s.io/v1 kind: CustomResourceDefinition metadata: name: flighttickets.flights.com
spec: scope: Namespaced group: flights.com names: kind: FlightTicket singular: flightticket plural: flighttickets
shortnames: - ft versions: - name: v1 served: true storage: true
```

Below is an example of the custom controller written in Go. This controller monitors and synchronizes the

```
```go theme={null}
package flightticket

import "k8s.io/api/apps/v1"

var controllerKind = v1.SchemeGroupVersion.WithKind("Flightticket")

// Run begins watching and syncing.
func (dc *FlightTicketController) Run(workers int, stopCh <-chan struct{}) {}

// callBookFlightAPI invokes the Book Flight API for a ReplicaSet.
func (dc *FlightTicketController) callBookFlightAPI(obj interface{}) {}
```

To deploy the operator, simply run:

```
```bash theme={null}
```

Deploy the operator

```
kubectl create -f flight-operator.yaml
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

The operator framework not only streamlines resource deployment but also simplifies ongoing management.

```
</Callout>
```

One of the most popular examples is the etcd operator. It deploys and manages an etcd cluster within Kubernetes.

```
<Frame>
```

! [The image illustrates an Operator Framework with Custom Resource Definitions (CRD) and Custom Controllers.]

```
</Frame>
```

Kubernetes operators handle tasks that would typically require manual intervention by system administrators.

For a comprehensive list of available operators, visit the [Operator Hub](https://operatorhub.io/). Many popular operators are available.

How to Deploy an Application Using an Operator

Deploying an application with an operator is an easy process that typically involves:

1. Installing the Operator Lifecycle Manager.
2. Deploying the operator.
3. Enjoying streamlined application management.

The following commands show you how to install the Operator Lifecycle Manager and deploy the etcd operator.

```
```bash theme={null}
```

```
Install the Operator Lifecycle Manager
```

```
curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.18.3/install.sh
```

```
Deploy the etcd operator
```

```
kubectl create -f https://operatorhub.io/install/etcd.yaml
```

```
Retrieve the installed Cluster Service Version in the "my-etcd" namespace
```

```
kubectl get csv -n my-etcd
```

This overview provides a high-level understanding of how operators simplify application management. A deep dive into operators will be explored in a dedicated future lesson. For exam preparation, note that most content primarily focuses on CRDs, making this article a valuable supplemental resource.

Thank you for reading, and we'll see you in the next lesson.